

VIVE追踪器（3.0） 开发者指南

Ver. 1.1

版本说明

版本号	发布日期	内容
1.0	2021.01.18	VIVE Tracker (3.0) 的首个公开版本
1.1	2021.05.27	更新GPIO的绝对最大额定值和电气特性

©2021 HTC Corporation.保留所有权利。HTC、HTC徽标、VIVE、VIVE徽标以及所有其他HTC产品和服务名称均为HTC Corporation及其附属公司在美国和其他国家的商标或注册商标。

目录

介绍	1
使用场景	1
硬件要求	3
界面	3
射频 (RF)	4
权力	5
光学	5
入船坞	6
机械方面的考虑	9
服装尺码	9
主要功能	10
对接机制	11
阻尼机构	12
配件设计	13
坐标系	18
软件组件	20
系统要求	20
数据格式	21
配件集成	24
Unity集成	28
VIVE跟踪器和VIVE角色	40
在Unity或Unreal中跟踪	42
固件升级	42

介绍

本文件阐述了VR配件制造商和内容开发者的发展指南。其中包含如何使用VIVE Tracker (3.0) 来实现位置跟踪和特定数据传输的信息 (无论是否使用HTC VIVE VR系统)。

VIVE Tracker (3.0) 可与HTC无线适配器配对，或通过USB接口将追踪数据传输至电脑。可连接至VIVE Tracker (3.0) 的配件可：

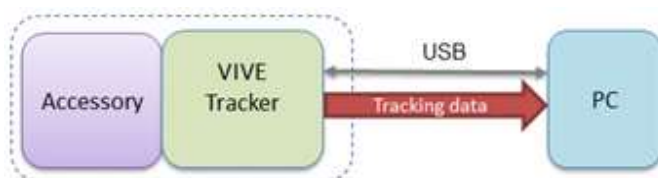
通过底层Pogo引脚接口模拟VIVE控制器按键功能。

通过VIVE Tracker (3.0) 的USB接口向PC发送特定数据。

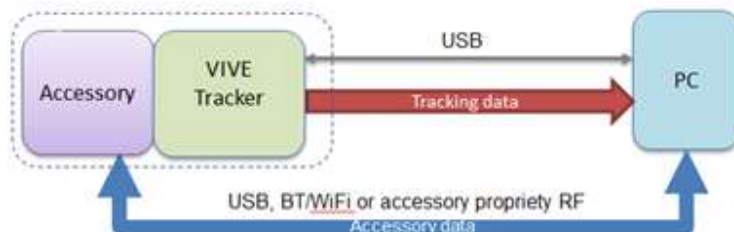
使用场景

VIVE Tracker (3.0) 支持五种使用场景。

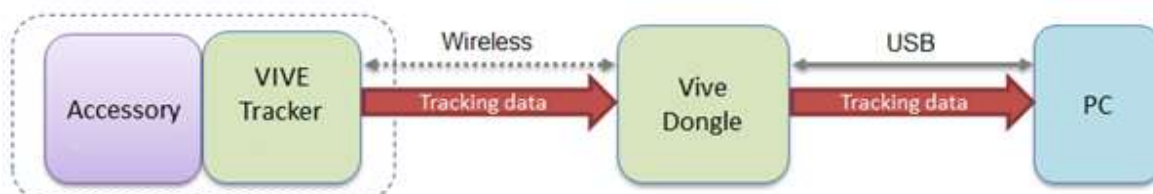
用例1：在VR中通过USB接口追踪被动物体。此情况下无需使用dongle，VIVE Tracker (3.0) 通过USB连接到PC，直接传输追踪数据。



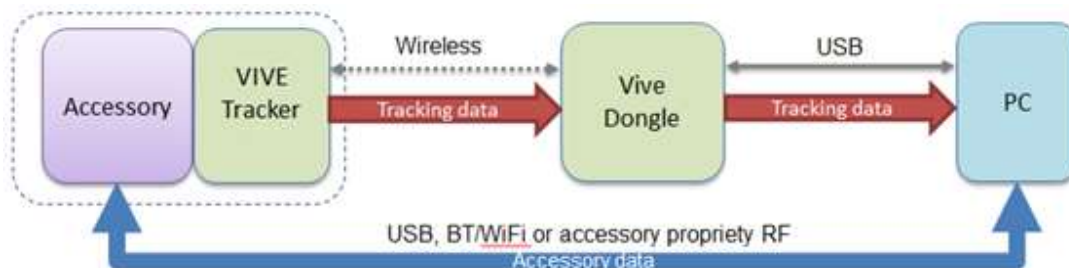
用例2：在VR中通过USB接口追踪被动物体，配件通过USB、BT/Wi-Fi或专有RF将数据传送到PC。这与用例1类似，但附件会根据其设计，将跟踪数据直接传输到PC以实现特定目的。



用例3：在VR中通过无线接口跟踪移动物体。在此用例中，使用dongle将跟踪数据从VIVE Tracker（3.0）传输到PC。



用例4：在VR中使用无线接口跟踪移动物体，配件通过USB、BT/Wi-Fi或专有RF将数据传递给PC。这与用例3类似，但配件根据其设计将跟踪数据直接传递给/来自PC，以实现特定目的。



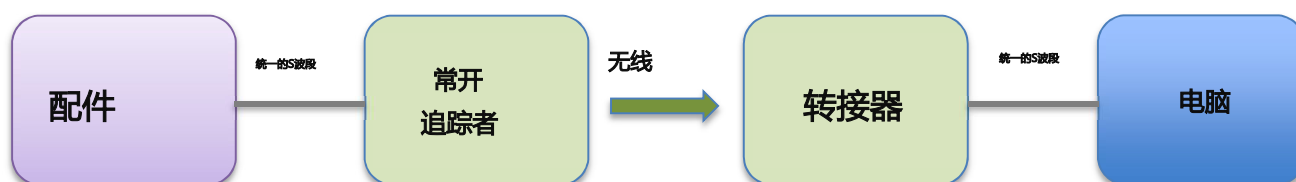
用例5：在VR环境中通过无线接口追踪移动物体，该配件可模拟VIVE控制器按键或通过VIVE追踪器（3.0）向PC传输数据。该方案与用例3类似，但配件通过Pogo引脚或USB接口与VIVE追踪器（3.0）连接，将按键事件传输至PC。



硬件要求

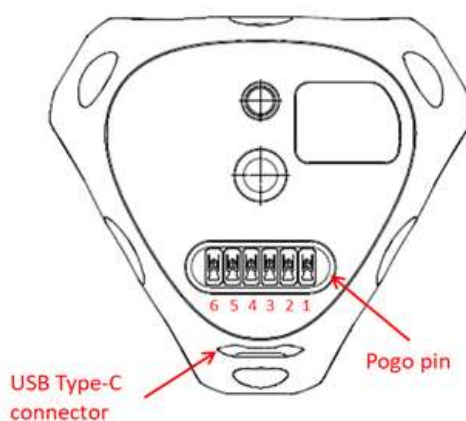
本节描述了与VIVE Tracker (3.0) 一起使用的配件的硬件要求，以便为HTC VIVE VR系统启用位置跟踪和特定数据输入。

兼容配件可连接至VIVE Tracker (3.0)，通过其USB接口将特定数据传输至电脑。需先将VIVE Tracker (3.0) 与适配器配对，才能将数据传输至电脑。下图展示了该系统的架构设计。



界面

USB Type-C接口支持全速USB 2.0 (客户端)。



GPIO绝对最大额定值的百分比

符号	参数	分钟	最高的	单元
我	输入电压	- 0.3	4	V
V_{ESD}	静电放电电压，人体模型	--	4000	V

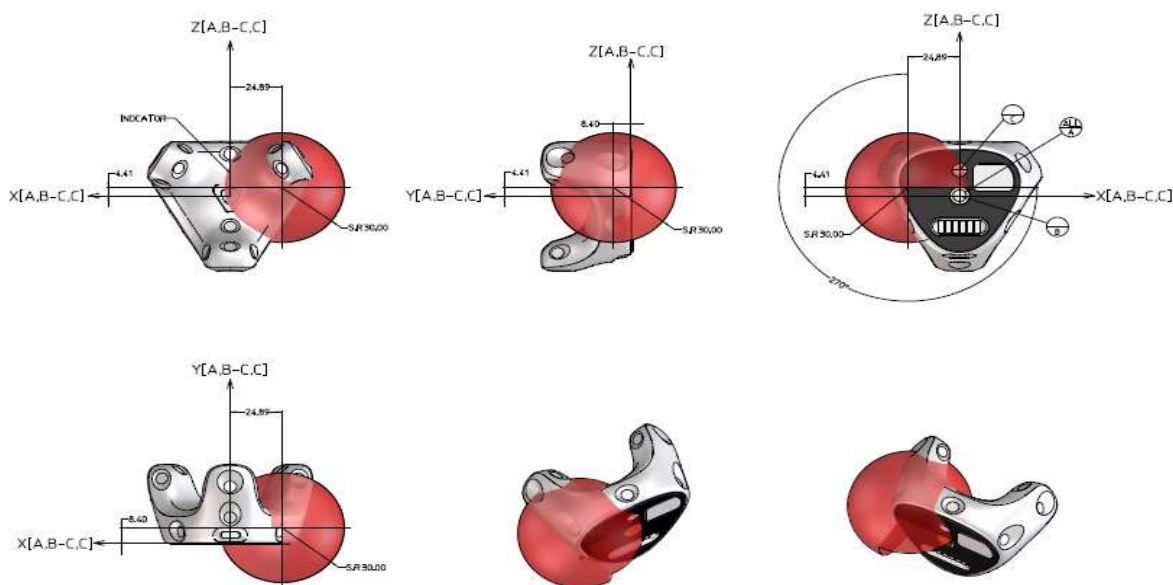
GPIO接点电气特性 (电源电压VDD=3.3 V)

符号	参数	条件	分钟	类型	最高的	单元
验证摘机	高压输出电压	$I_{OH} = -4\text{mA}$	VDD - 0.9	2.8	心房同步心室抑制型起搏	V
		$I_{OH} = -16\text{mA}$	VDD - 1.4	2.3	心房同步心室抑制型起搏	V
卷	低电平输出电压	$I_{OL} = 4\text{mA}$	--	0.2	0.4	V
		$I_{OL} = 16\text{mA}$	--	0.7	0.9	V
暴力致残	高电平输出电流	--	0.7vdd	--	Vdd+0.3	V
Vil	低电平输出电流	--	--	--	0.7	V
I_{OH}	高电平输入电流	--	--	--	-16	妈
I_{OL}	低电平输出电流	--	--	--	16	妈
I_{IH}	高电平输入电流	--	-1	--	1	uA
I_{IL}	低电平输入电流	--	-1	--	1	uA

射频 (RF)

为确保VIVE Tracker (3.0) 与适配器之间建立稳定无线连接，当配件连接至VIVE Tracker (3.0) 时，其OTA性能衰减不得超过3dB。

以下是关于改善射频性能的建议。



图：展示“禁止进入”区域，该区域仅允许配件的非金属部件进入（球体半径为30毫米，中心为天线馈电点）

除1/4英寸螺丝、电连接垫（与Pogo针脚相连）及其相关电路等关键部件外，配件的金属部件与天线之间应保持至少30毫米间距，以避免在连接VIVE Tracker (3.0) 时影响OTA性能。

权力

USB-C连接器	电压要求	最大充电电流	充电时间（约）
AC	5V+/-5%	700 mA	2 hrs
电脑		500 mA	3 hrs
Pogo Pin 3	电压要求	最大充电电流	充电时间（约）
电脑	5V+/-5%	500 mA	3 hrs

笔记

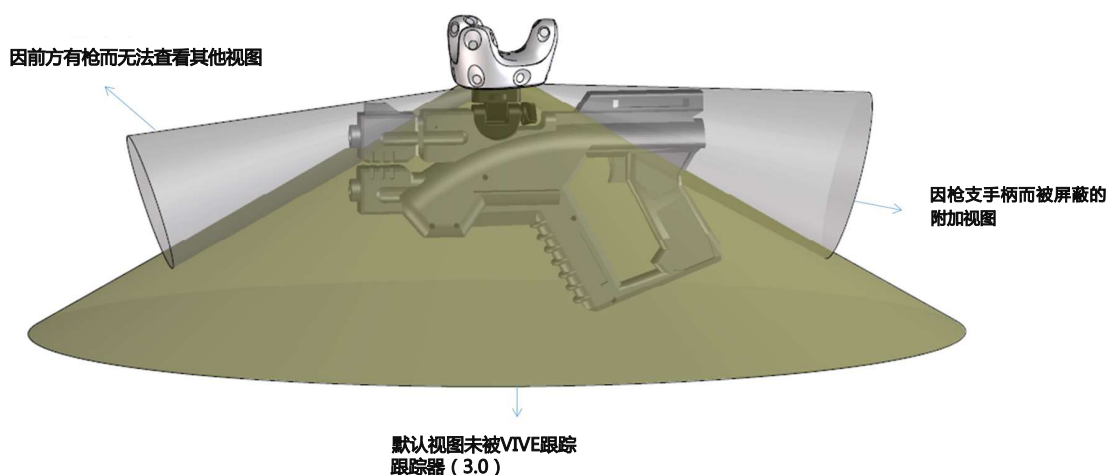
AC : D+短至D

PC : D+/D-通信

光学

VIVE Tracker (3.0) 的视野 (FOV) 为240度。避免遮挡VIVE Tracker (3.0) 的FOV，因为这会阻塞跟踪器传感器的响应。

若对接部件超出推荐放置锥体范围，将无法显示更多视图。



图：部分对接柱延伸超出推荐放置锥体

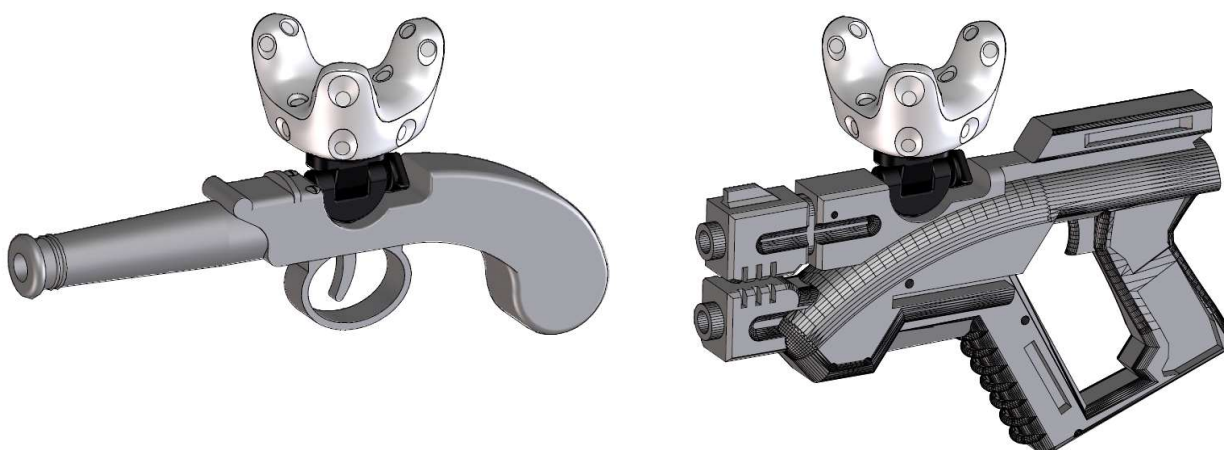
入船坞

对接兼容性要求：

- VIVE Tracker（3.0）的对接设计遵循ISO标准（ISO 1222：2010）。此外，VIVE Tracker（3.0）存在一些限制，例如较长的螺钉无法完全旋入。
- 用户应能轻松地用双手安装和拆卸VIVE Tracker（3.0）。一只手拿着VIVE Tracker（3.0），另一只手拿着配件。
- 用户在连接或断开VIVE Tracker（3.0）时不应面临身体伤害风险。
- 用户在连接和断开VIVE Tracker（3.0）时应感到舒适。
- VIVE Tracker（3.0）的配件应采用实体造型设计，以避免使用时发生碰撞。
- 配件不应阻塞VIVE Tracker（3.0），以免影响跟踪性能。
- 强烈建议配件采用低反射材料制作外壳，以避免与追踪传感器产生反射干扰，尤其当配件需安装在VIVE Tracker（3.0）的FOV内部时。

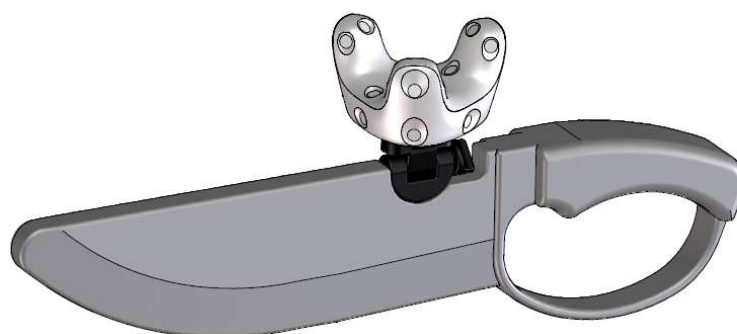
对接实施例

枪



剑

建议将安装机构靠近持剑位置，并在VR应用中设置剑的长度。



多功能对接基座

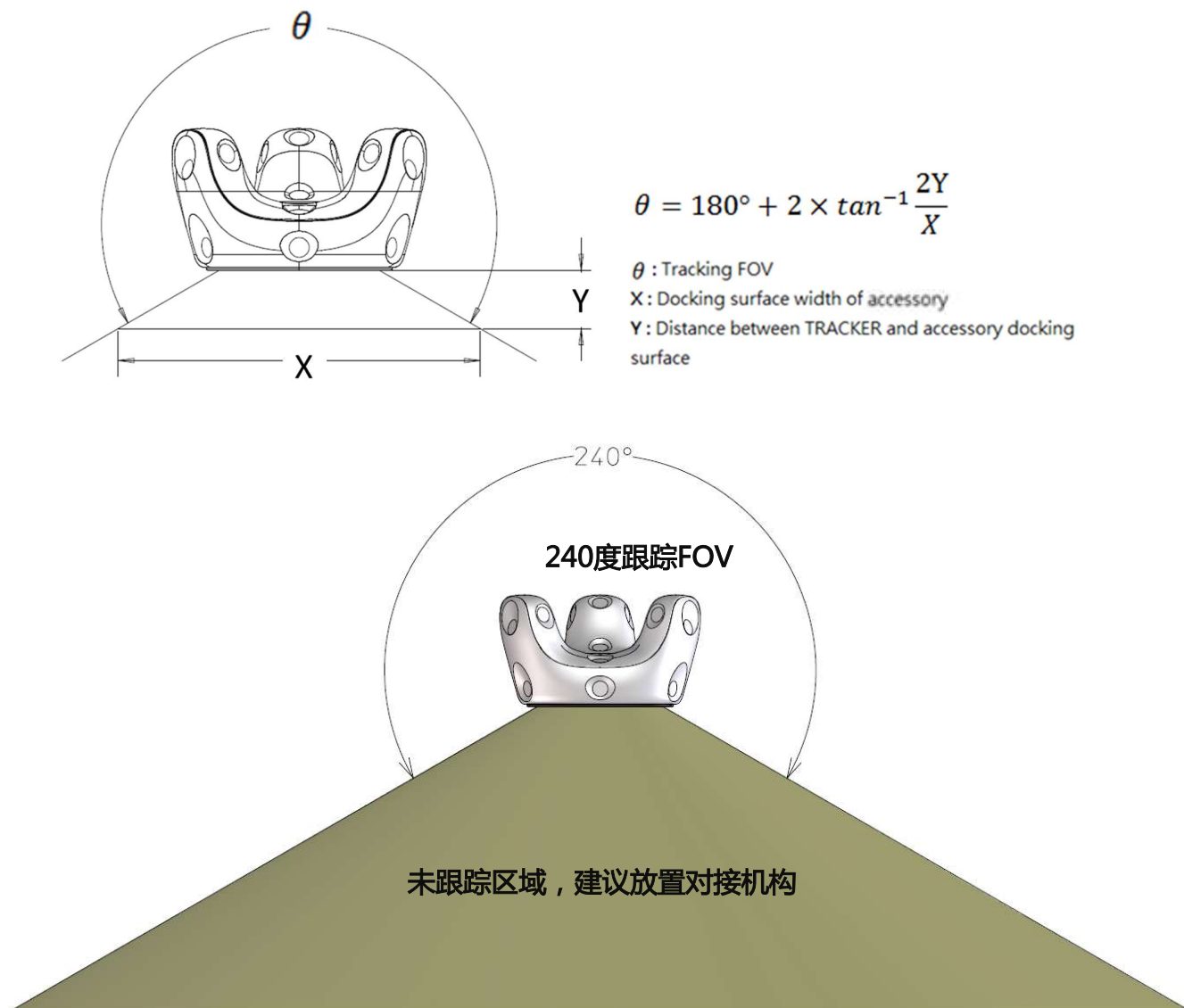
用户可将VIVE Tracker (3.0) 附着于任何需要追踪的物体或表面。

若物体表面光滑且坚硬，建议使用强力胶带（如3M VHB胶带）固定对接基座。

若物体表面粗糙或柔软，建议使用绑带将对接底座固定在该表面。



VIVE Tracker (3.0) 若安装不当，可能导致配件部分遮挡追踪器的FOV，从而影响追踪效果。追踪器FOV与相关配件的安装间距如下所示：

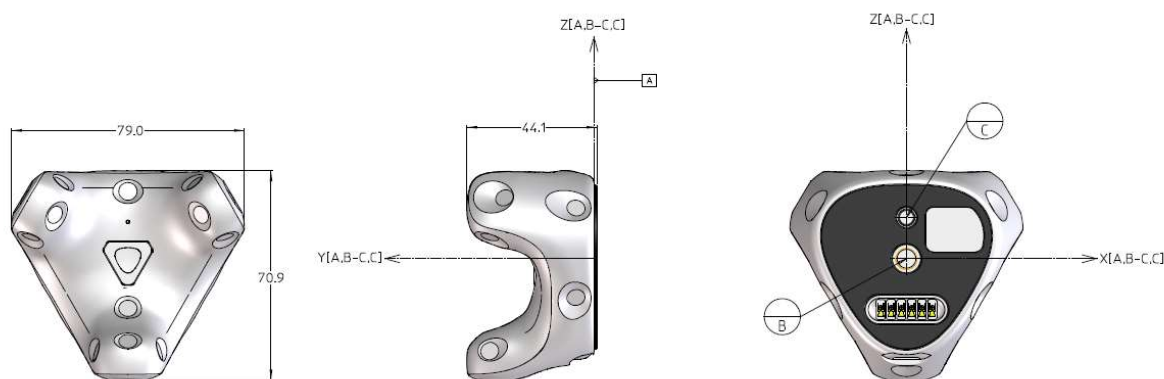


机械方面的考虑

本节阐述了开发者在开发适配VIVE Tracker (3.0) 的各类配件时需考虑的机械设计要点。



尺寸和重量

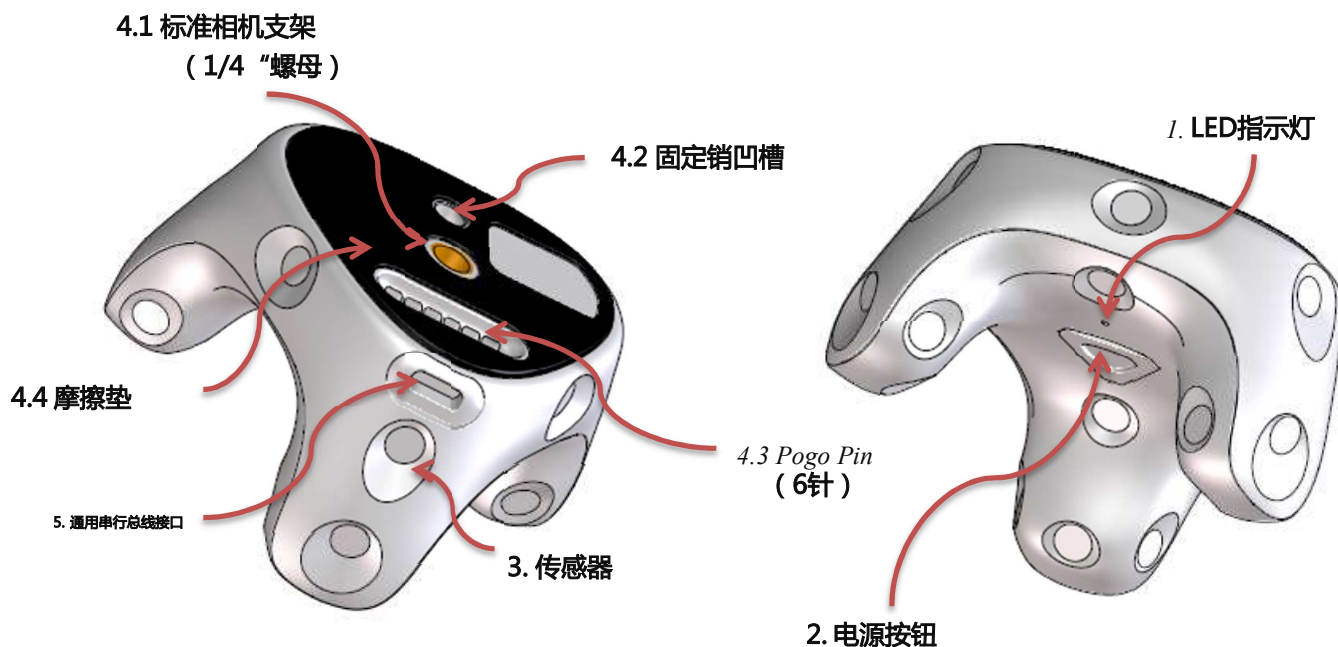


VIVE Tracker (3.0) 的不同角度

尺寸：79.0mm (长) × 70.9mm (宽) × 44.1mm (高)

重量：75克

主要功能



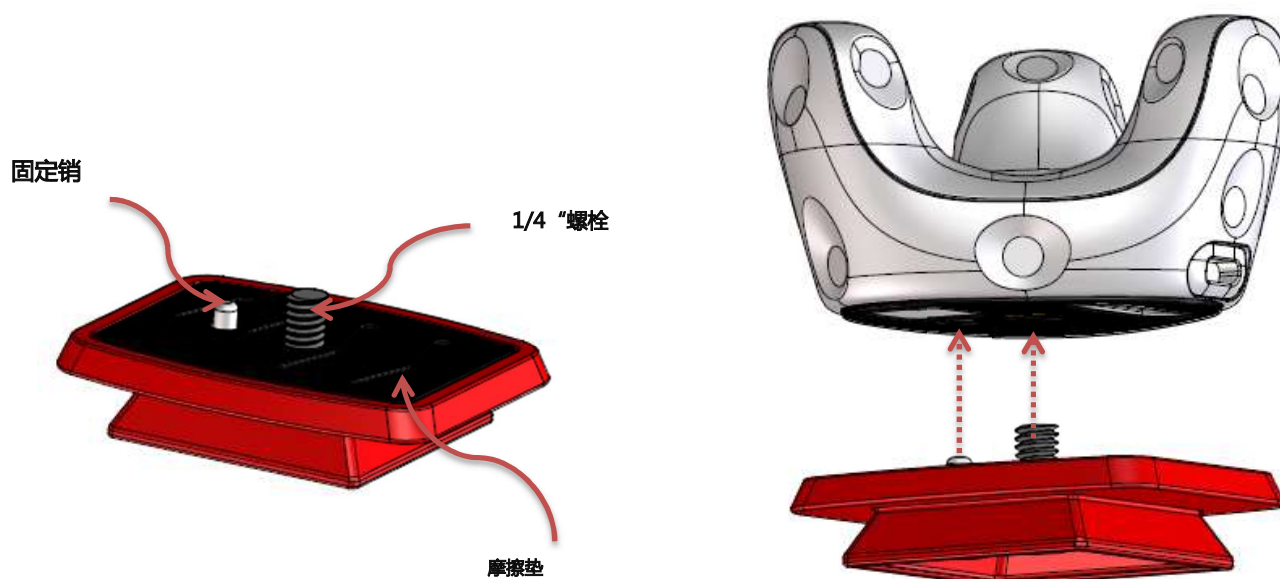
1. LED指示灯：显示VIVE Tracker (3.0) 的状态。
2. 电源按钮：用于通电/断电、BLE配对等。
3. 传感器：接收来自基站的信号。VR系统使用接收信号来计算VIVE Tracker (3.0) 的当前位置。配件应尽量减少表面反射（例如避免白色），因为这可能会导致信号故障并影响性能。建议使用防反射涂料。
4. 对接机制：采用标准相机三脚架对接方式，具体包括：
 - 4.1 1/4英寸螺钉螺母用于固定配件。
 - 4.2 稳定销钉凹槽以防止跟踪器旋转。
 - 4.3 Pogo pin端口（弹簧触点式），用于可选配件的电气连接。
 - 4.4 摩擦垫，用于在配件与VIVE Tracker (3.0) 之间提供摩擦表面
5. USB端口：通过USB Type-C电缆与配件进行电气连接

对接机制

VIVE Tracker (3.0) 采用通用相机三脚架对接方法，遵循ISO标准 (ISO 1222 : 2010) 。

以下是Tracker (3.0) 如何安装在配件上的示意图。

与标准三脚架托架头对接



VIVE Tracker (3.0) 可先安装在头枕上，再固定到配件主体上（类似于相机安装在三脚架上的方式）。

阻尼机构

在研究和实际使用中发现，VIVE Tracker (3.0) 的持续振动会影响IMU性能，导致明显的IMU漂移。为解决这个问题，建议在对接机构中使用阻尼系统。下图展示了如何利用现有阻尼橡胶（作为无人机配件购买）来实现这一目的。



在确定VR内容拟采用的振动参数时，需重点调整阻尼橡胶的硬度、安装间距/位置以及橡胶材质等关键要素。

配件设计

以下是符合ISO标准的不同附件机制：

.1/4英寸螺栓设计

请参见ISO 1222-2010，第1页图1。

.稳定销设计

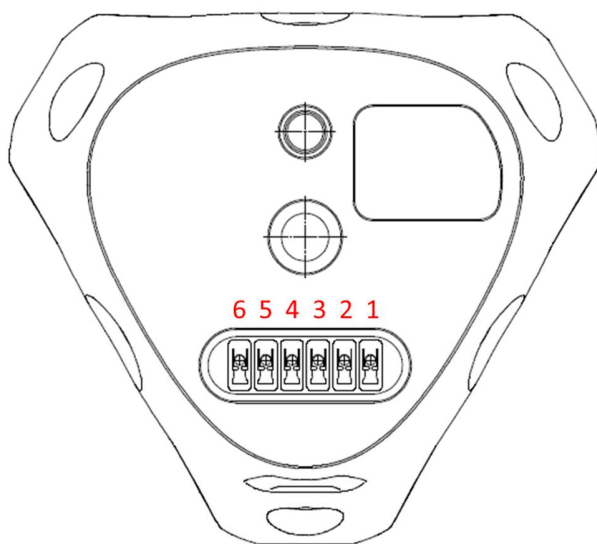
VIVE Tracker (3.0) 采用ISO 1222-2010标准第3页图5所示设计，具体尺寸与公差参数详见第13-17页。建议使用稳定销以提升追踪性能。

螺纹设计

VIVE Tracker (3.0) 采用1/4英寸螺纹，螺距为1.27毫米。详细信息请参见ISO 1222-2010标准第3-5页。

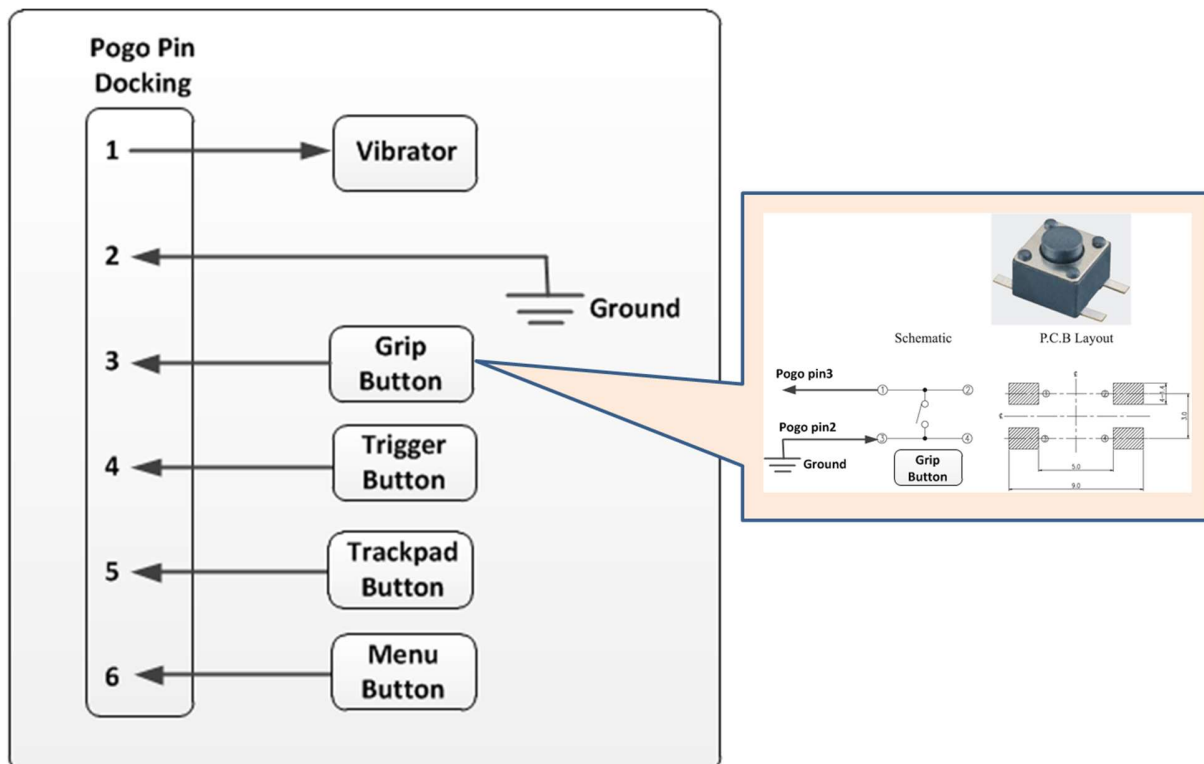
Pogo Pin设计

a. VIVE Tracker (3.0) 的针头定义



b. Pogo Pin参考设计

电气

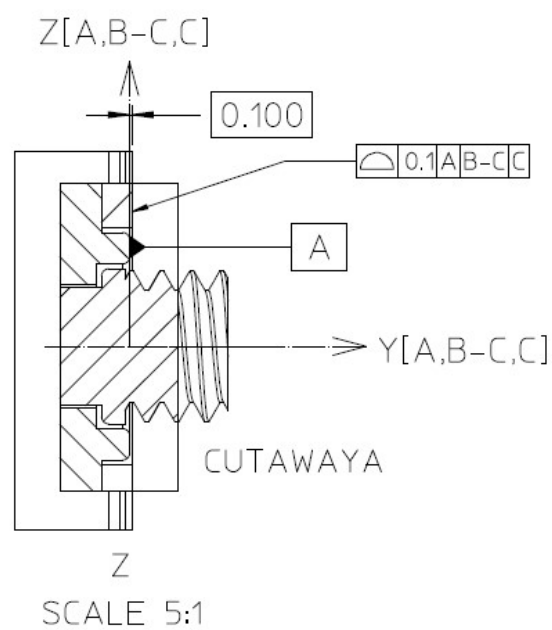
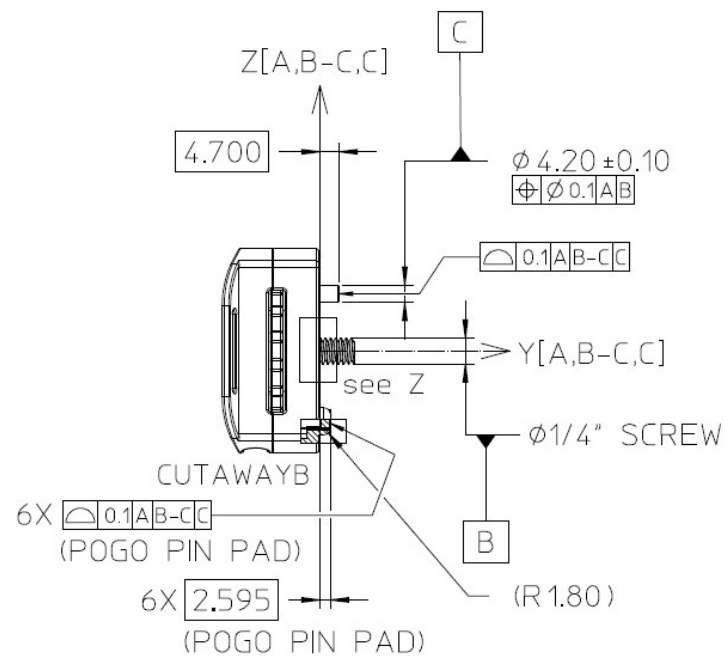


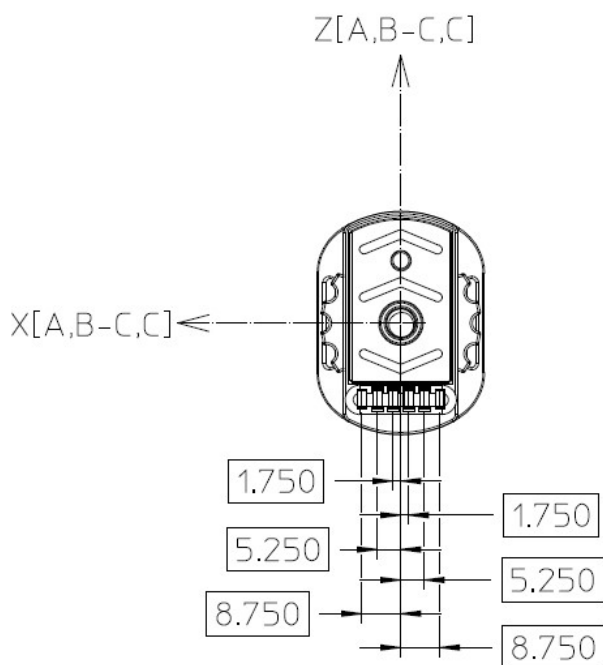
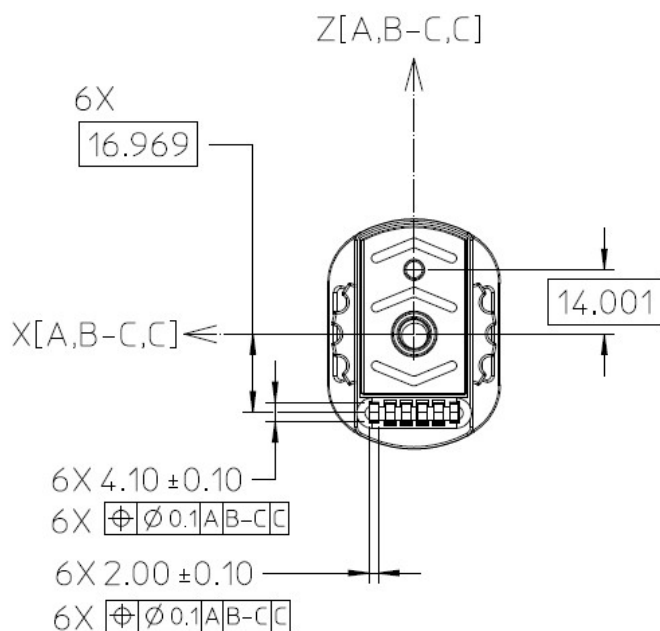
钉 不	类型	描述
1	数字输出	通用输出引脚
2	GND	地面
3	数字/电源 输入	1. 通用输入引脚：内部上拉电阻至VDD，低电平有效（握把按钮） 2. 电源输入引脚
4	数字输入	通用输入引脚：内部上拉电阻至VDD，低电平有效（触发按钮）
5	数字输入	通用输入引脚：内部上拉电阻至VDD，低电平有效（触控板按钮）
6	数字输入	通用输入引脚：内部上拉电阻至VDD，低电平有效（菜单按钮）

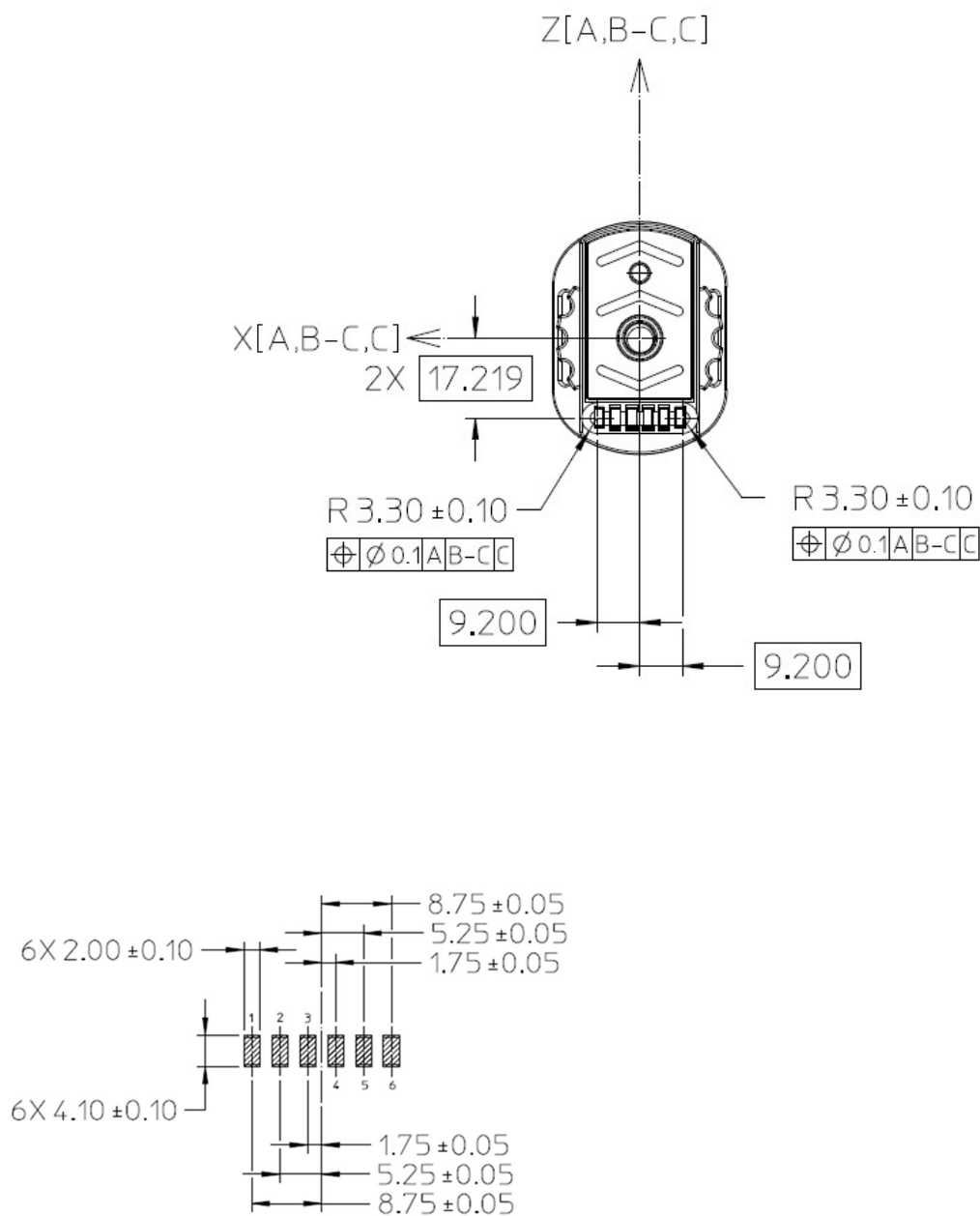
USB Type-C
connector

Pogo pin

机械







Contact Resistance :

30m Ω (Max) initial when measured at 20mV (Max) open circuit at 100mA.

Contact Current Rating :

1.8A (Min)

坐标系

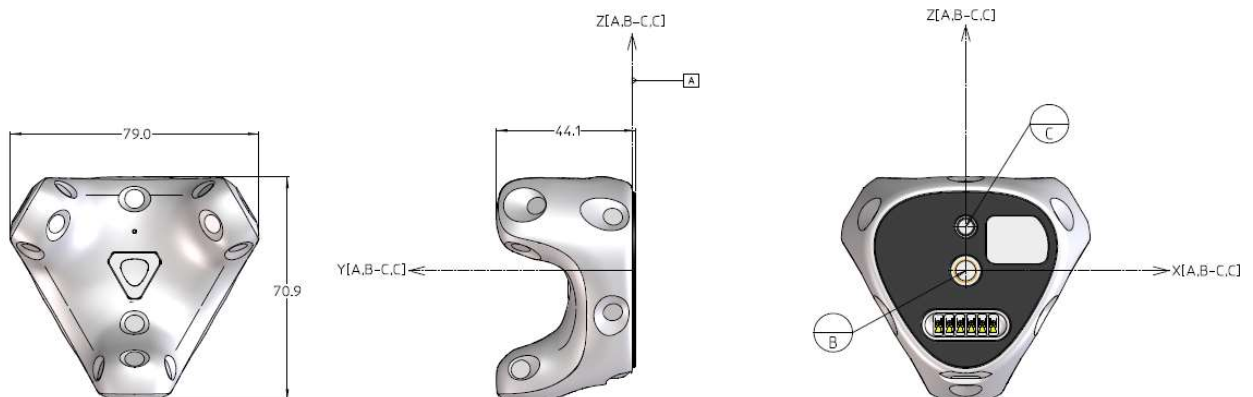
VIVE Tracker (3.0) 采用 “右手坐标系” 。

日期A设定为1/4英寸螺母环形结构的顶部表面。

基准B被设定为标准相机支架 (1/4英寸螺栓) 中心线与基准A的交点。

基准面C设定为稳定销凹槽中心线与基准面A的交点。

坐标系由基准A的基准框、基准B和基准C的连线以及基准C本身构成。



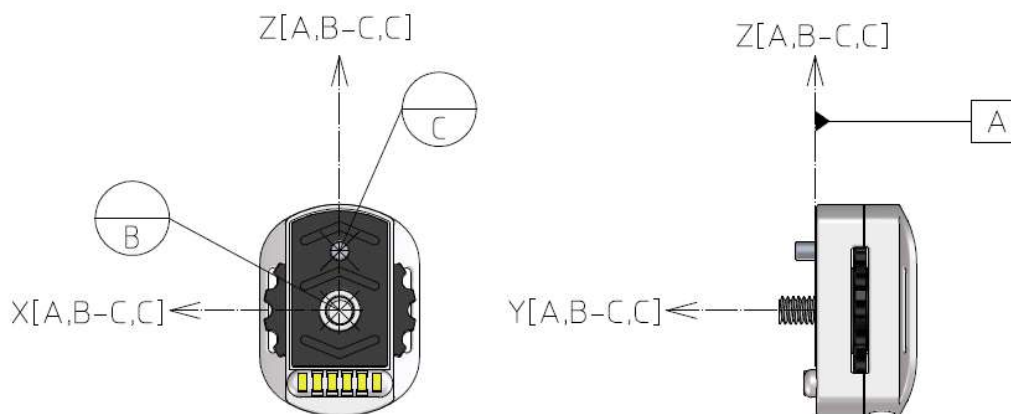
附件：

将日期A设定为1/4英寸螺栓周围环形特征的顶面。

基准B设定为1/4英寸螺钉中心线与基准A的交点。

基准面C设定为稳定销中心线与基准面A的交点。

坐标系由基准A的基准框、基准B和基准C的连线以及基准C本身构成。



软件组件

本节描述VIVE Tracker (3.0) 的软件组件。

若你是配件制造商，可通过VIVE Tracker (3.0) 的Pogo针脚端口传输按钮事件。关于配件与VIVE Tracker (3.0) 之间的详细数据格式传输方式，请参阅数据格式章节。

如果您是内容开发者，请参阅Unity集成。

配件集成。

当VIVE Tracker (3.0) 的新固件发布时，您可以通过USB Type-C线缆将追踪器连接到PC来更新固件。了解如何在固件更新中更新固件。

系统要求

对于内容开发者和配件制造商来说：

1. 若要测试VIVE Tracker (3.0) 与您的内容或配件，您需要配备HTC VIVE以及运行所需的硬件和软件。
2. 您需要一台至少有一个可用USB 2.0端口的电脑，用于插入适配器（适用于使用案例中提到的适配器）或VIVE Tracker (3.0) （用于更新固件）。该电脑还应安装SteamVR。

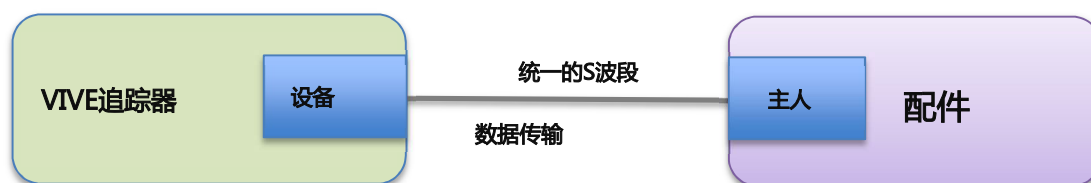


配件制造商：

若您的配件需通过VIVE Tracker (3.0) 模拟VIVE控制器的按钮信号至PC，必须支持以下接口：

-Pogo pin：请参阅硬件要求部分。

-USB接口：支持全速USB主机和HID类。VIVE追踪器 (3.0) 将作为USB设备，用于与外接配件进行数据传输。



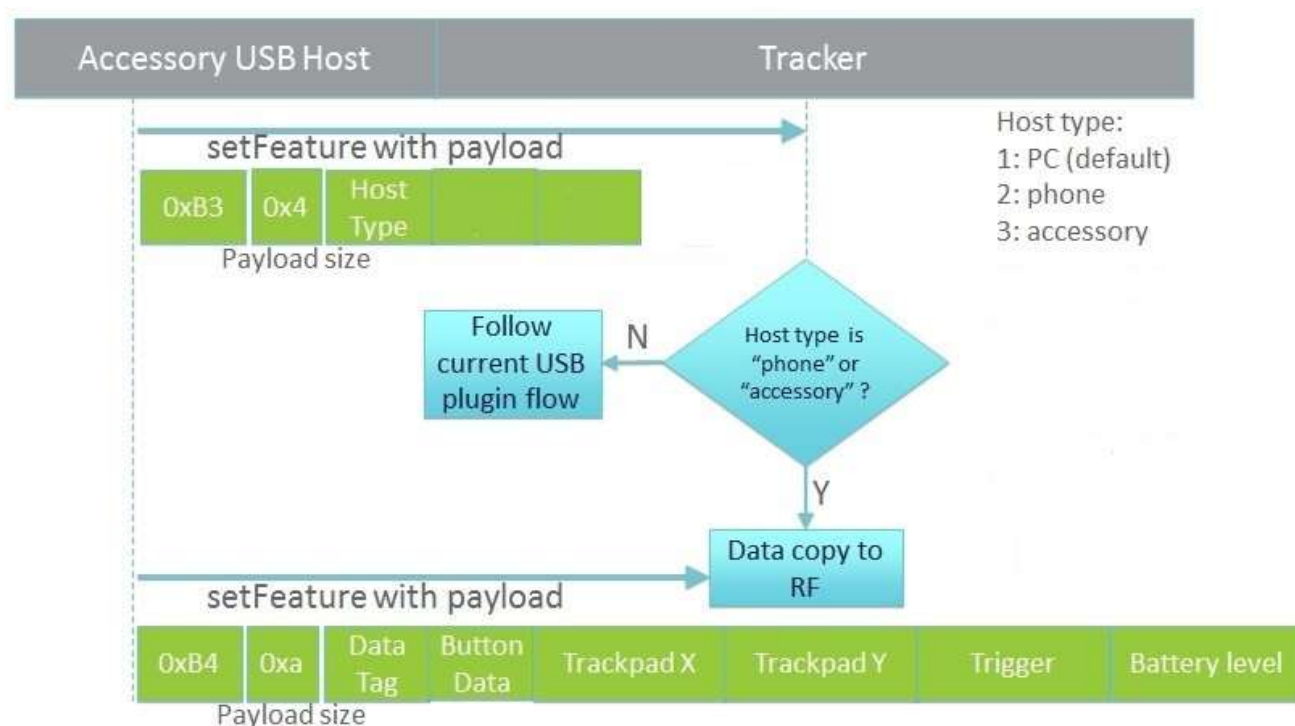
数据格式

本节说明配件制造商在使用USB接口时，可通过VIVE Tracker (3.0) 在配件与PC之间传输数据时可采用的数据格式。

从配件传输到VIVE Tracker (3.0) 的数据格式通过USB HID功能报告发送，其与VIVE控制器的用户界面类似，发送数据的间隔应大于10ms。

若需深入了解USB HID规范，请访问[_USB_官网获取详细信息](#)。当VIVE Tracker (3.0) 作为USB设备连接至PC时，将显示三个USB节点。其USB VID/PID参数设置为28DE/2300.0)，并确认接口名称已更改为 'Controller'。将windex设置为2，以便向VIVE Tracker (3.0) 发送USB指令。

请参阅下表，了解配件与VIVE Tracker (3.0) 之间的USB命令流程。



设置特征0xB3数据格式

字节索引	数据	备注
0	主机类型	2 电话 3 附件
1	N/A	已预订
2	N/A	已预订

设置特征0xB4数据格式

字节索引	数据	备注
0	标签索引	表示要发送的数据的版本。此数据格式的默认值为零。
1	按钮	触发器 0x01 保险杠 0x02 菜单 0x04 水蒸气 0x08 补丁 0x10 PAD_FINGERDOWN 0x20保留 已预订 0x80
2	Pad X值	

字节索引	数据	备注
3	Pad Y值	Pad X值, 值从-32768到32767 字节2是LSB
4		Pad Y值, 值从-32768到32767 字节4是LSB
5		
6	原始触发	Trigger Raw, 数值范围为0到65535 字节6是LSB
7		
8	电池电量	电池电量, 保留 字节8是LSB
9		

表：数据格式，VIVE跟踪器 (3.0) 附件

示例代码

以下是向VIVE Tracker (3.0) 发送setFeature命令的示例代码。但您需要参考系统以获取正确的API来发送USB setFeature命令。

Visual Studio

```
buffer[0] = 0xB3;
buffer[1]=0x03 ; //长度
buffer[2]=0x03 ; //1 : PC , 2 : Phone , 3 : Accessory
buffer[3] = 0x01;
buffer[4] = 0x00;

if (!HidD_SetFeature(m_hDevice, buffer, sizeof(buffer))) {
    AfxMessageBox(L"Error: Failed to set feature.");
}
```

JAVA

```
//以0xB3命令为例
data1[0]= ( byte ) 179 ; //0xB3
data1[1]=3 ; //表示后面有3个字节
data1[2]=3 ; //主机类型：附件
data1[3] = 1; //Reserved
data1[4] = 1; //Reserved
int result=mDeviceConn.controlTransfer ( 0x21,0x09,0x0300,2 , data1 , data1 长度 ,
0 ) ;
```

STM F4系列开发板

```
char buffer[64];
char *p_buffer = buffer;
unsigned int缓冲区大小=缓冲区大小; memset ( ( void*) p_buffer, 0, 缓冲区大小 ); p_buffer[0]=0xB3; //命令p_buffer[1]=0x03; //数据大小p_buffer[2]=0x03; //主机类型：附件
p_buffer[3] = 0x01;
p_buffer[4] = 0x01;
//stm API
USBH_HID_设置报告 (
    拉手
    0x03, //功能报告
    0,
    (uint8_t *)p_buffer,
    buffer_size );
```

配件集成

Pogo输出引脚信号持续时间：

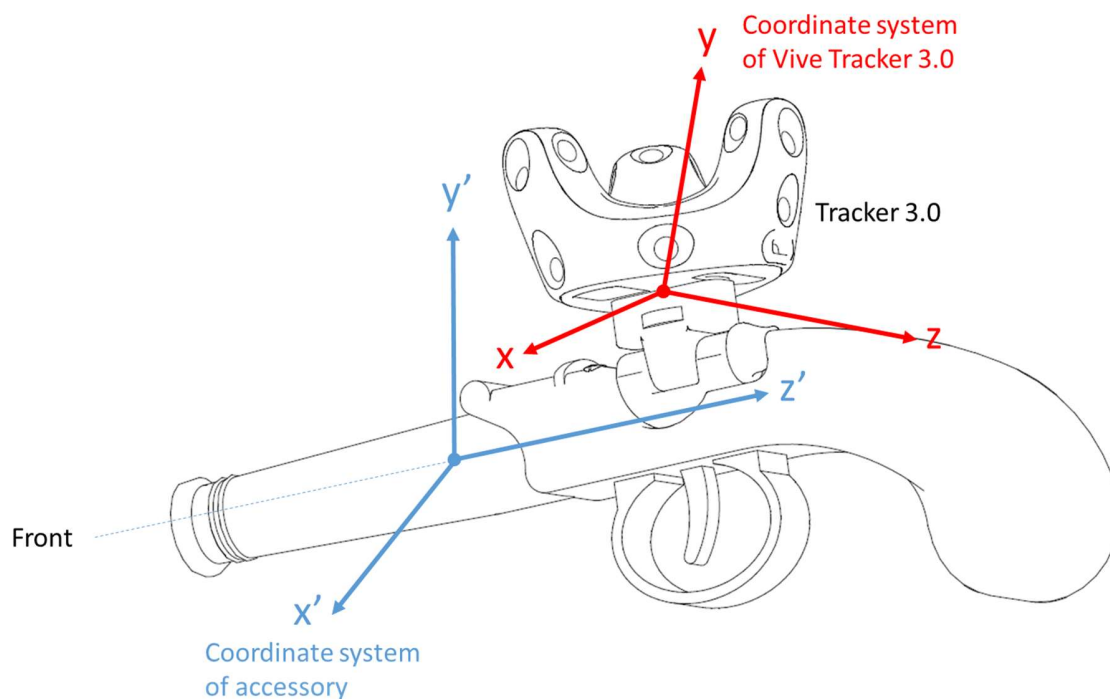
VIVE Tracker (3.0) 会从内容中接收触觉输入值，而Pogo输出引脚将以毫秒为单位输出持续时间值。

本节介绍配件与VIVE Tracker (3.0) 之间的位置转换信息。内容开发者可在Unity等游戏引擎中，为使用该配件的内容生成正确的旋转和平移效果。

假设配件的局部坐标系为z轴朝前（左旋坐标系），VIVE Tracker (3.0) 如下方示例所示安装在配件上。在积分过程中，VIVE Tracker (3.0) 相关配件的旋转角度和位移距离分别通过滚转角、偏航角、俯仰角以及D_x、D_y、D_z坐标进行描述。

在设计阶段确定配件中心后，可根据实际集成条件测量配件的以下尺寸和间距。有关VIVE Tracker（3.0）中心的详细信息，请参阅硬件和机械设计指南。

以下通过VIVE Tracker (3.0) 演示枪械配件的集成实例：



Pitch：绕x轴旋转的角度。

偏航：绕y轴旋转的角度。

滚动：绕z轴旋转的角度

D_x ：附件与追踪器在x轴方向的中心间距。

D_y ：附件与追踪器之间y 轴的中心距离。

D_z ：附件与追踪器之间z轴的中心距离。

内容开发者可收集上述信息，并将追踪器姿态转换为配件姿态。

假设跟踪器旋转矩阵为 $R_{Tracker}$ ，附件旋转矩阵 $R_{Accessory} = R_{pitch_yaw_Roll} * R_{Tracker}$ 。

且附件位置 $V_{Accessory} = V_{Tracker} + R_{Accessory} * 距离$

以下是Unity的示例代码 (参考) :

```
public class Accessory : MonoBehaviour {  
    const float dX = 0.0100224f;  
    const float dY = -0.07616526f;  
    const float dZ = 0.4884118f;  
  
    const float roll = 10.854305f;  
    const float yaw = 91.8736f;  
    const float pitch = 78.805113f;  
  
    void Update () {  
        //采集跟踪器与附件之间的增量旋转和位移  
        Vector3 delta_displacement = new Vector3(dX, dY, dZ);  
        四元数delta旋转等于四元数乘以欧拉旋转矩阵 ( 包含横滚、偏航、俯仰三个方向的旋  
转 ) ;  
  
        //获取当前跟踪器姿态  
        Vector3 tracker_position =  
SteamVR控制器输入(3).transform.pos ;  
        四象限陀螺仪旋转跟踪器=SteamVR控制器输  
入(3)的旋转变换 ;  
  
        //将当前跟踪器姿势转换为配件姿势  
        GameObject.Find ( "Accessory" ).transform.rotation=trac-  
ker_rotation*delta_rotation ;  
        GameObject.Find ( "Accessory" ).transform.position=trac-  
ker_position+ ( tracker_rotation*delta_rotation ) *  
delta_displacement;  
    }  
}
```

另一个Unity示例代码展示了如何通过比较VIVE Tracker (3.0) 的y轴和z轴向量 (示例中的AxisY_Tracker、AxisZ_Tracker) 与配件的y轴和z轴向量 (示例中的AxisY_Accessory、AxisZ_Accessory) 来实现配件的变换。

```
public class Accessory: MonoBehaviour {

    const Vector3 AxisY_Tracker = new Vectors(AxisY_Tracker_X,
AxisY_Tracker_Y, AxisY_Tracker_Z);
    const Vector3 AxisZ_Tracker = new Vectors(AxisZ_Tracker_X,
AxisZ_Tracker_Y, AxisZ_Tracker_Z);

    const Vector3 AxisY_Accessory = new Vectors(AxisY_Accessory_X, AxisY_
Accessory_Y, AxisY_Accessory_Z);
    const Vector3 AxisZ_Accessory = new Vectors(AxisZ_Accessory_X,
AxisZ_Accessory_Y, AxisZ_Accessory_Z);

    void Update () {

        //通过比较追踪器与附件的Y轴平行向量来计算增量旋转
        四元数delta_y轴角=四元数.从到旋转 ( Y轴追踪器 , Y轴辅助 ) ;
        AxisZ_Tracker = delta_rotY * AxisZ_Tracker;
        四元数delta_rotZ=旋量.FromToRotation ( AxisZ_追踪器 , AxisZ_附件 ) ;

        //采集追踪器与附件之间的增量旋转和位移数据
        Vector3 delta_displacement = new Vector3(dX, dY, dZ);
        角速度的Δz分量等于Δz与Δy的乘积 ;

        //获取当前跟踪器姿态
        Vector3 tracker_position      =
SteamVR控制器输入(3)位置变换 ;
        四象限旋转跟踪器=
SteamVR控制器输入(3)的变换参数为旋转 ;

        //将当前跟踪器姿势转换为配件姿势
        GameObject.Find ( "Accessory" ).transform.rotation=delta_rotation*tracker_
rotation ;
        游戏对象。查找 ( "附件" ) 。变换。位置=跟踪器位置+ ( 旋转
增量*跟踪器旋转 ) *
        delta_displacement;
    }
}
```

Unity集成

本节通过Unity游戏引擎，为内容开发者提供在VR内容中启用VIVE Tracker (3.0) 的示例。

首先，需要SteamVR检测到VIVE Tracker (3.0)。假设你已将两个VIVE控制器连接到适配器，并将适配器底座连接到电脑，右键单击控制器图标，在菜单中点击配对控制器，然后按VIVE Tracker (3.0) 的电源键2秒进入配对模式。



VIVE Tracker (3.0) 与dongle成功配对后，您将在SteamVR窗口中看到VIVE Tracker图标。



建议使用Unity 2017.3或更高版本（最低版本5.4.6），最新版本可在此处[下载](#)。

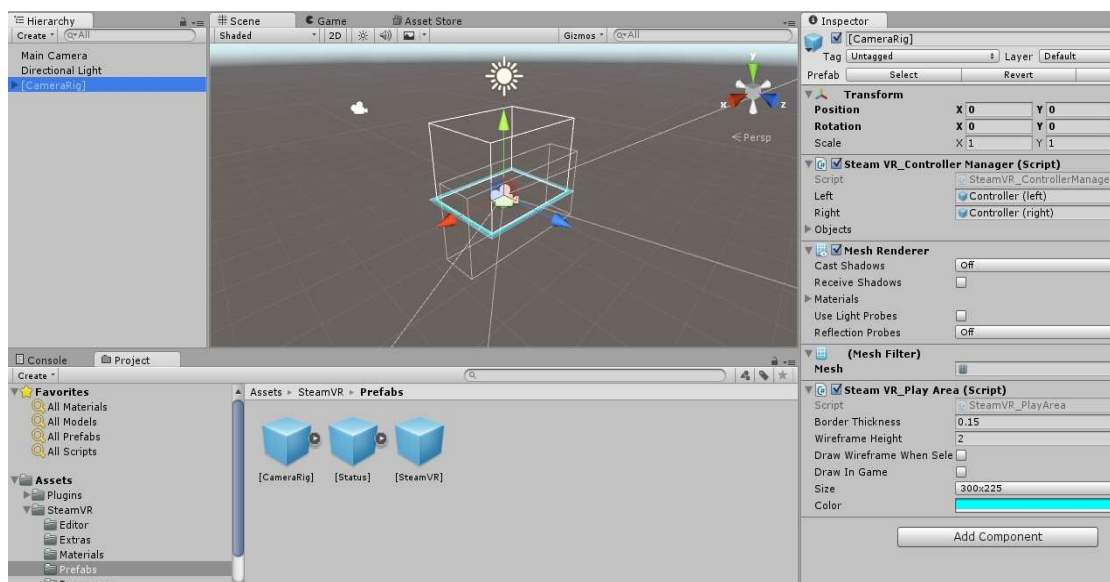
使用SteamVR插件

将SteamVR插件导入到项目中。如果尚未安装该插件，请从Unity资源商店下载。

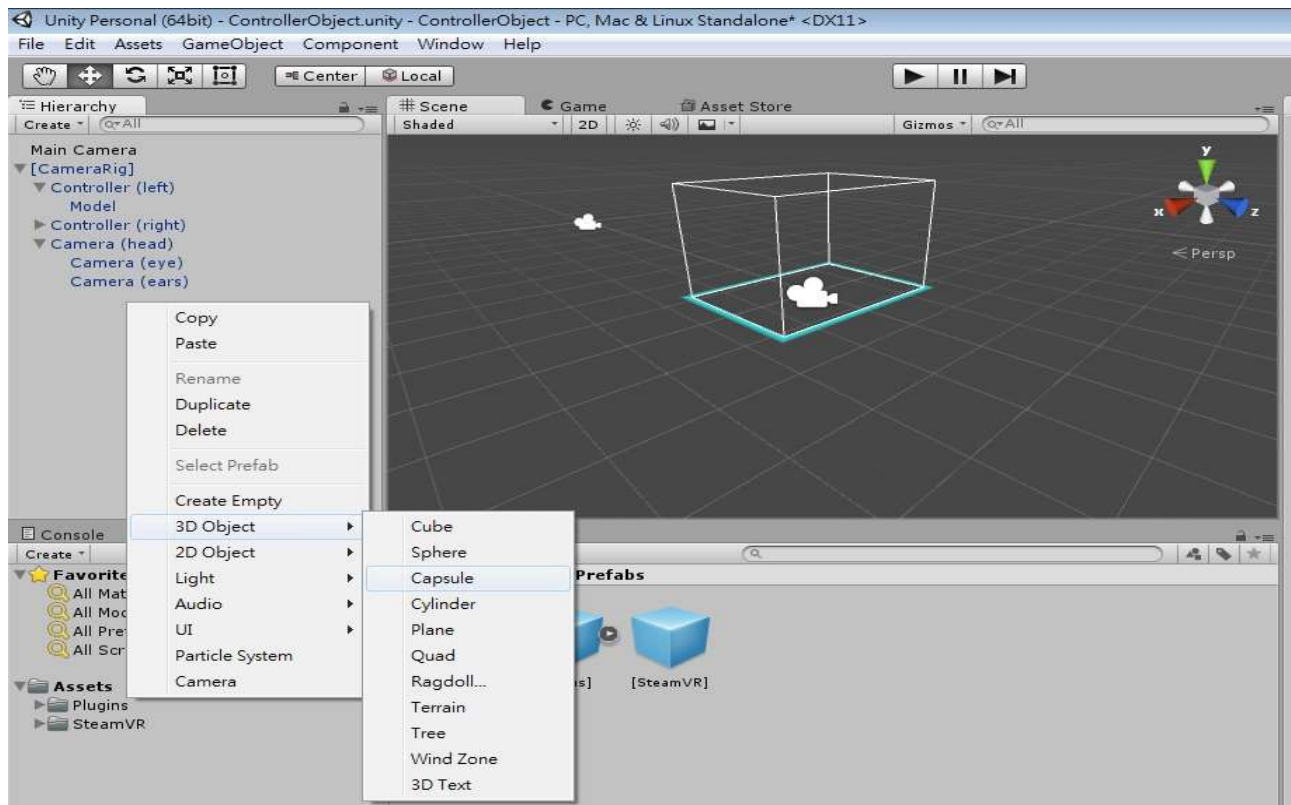


VIVE Tracker (3.0) 的开发者版本采用了与VIVE控制器内容创建相似的方法和命名方式。请按照以下步骤在Unity中为VIVE Tracker (3.0) 创建内容。

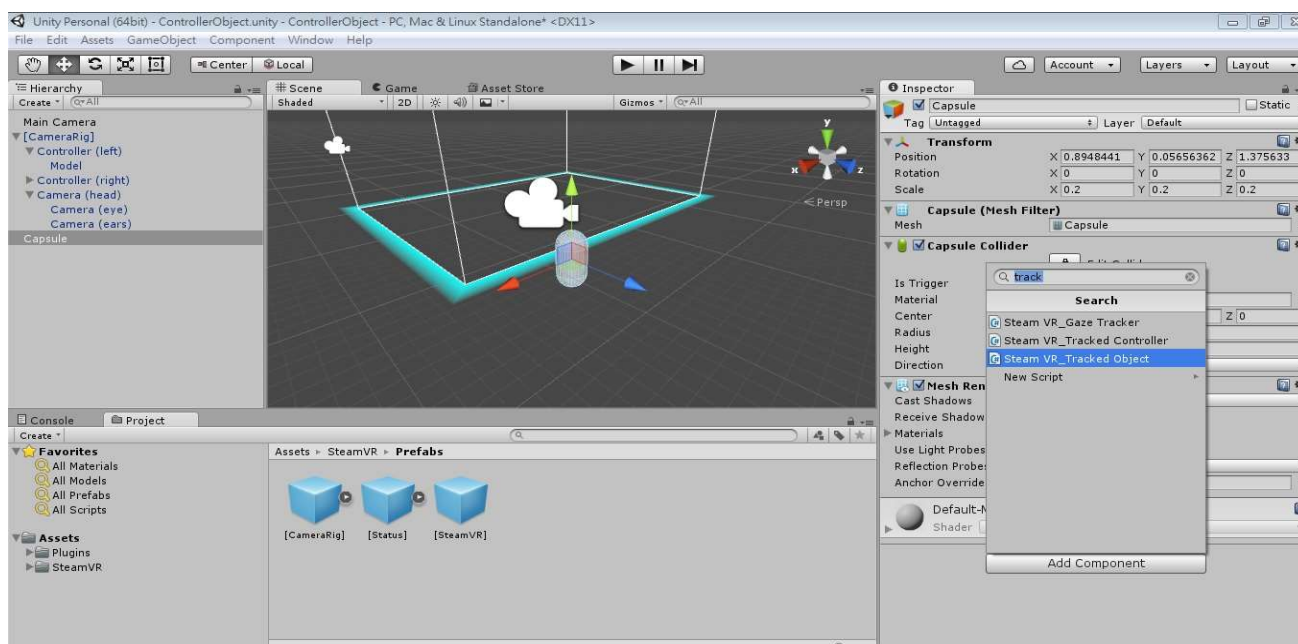
步骤1：将“CameraRig”添加到Hierarchy。



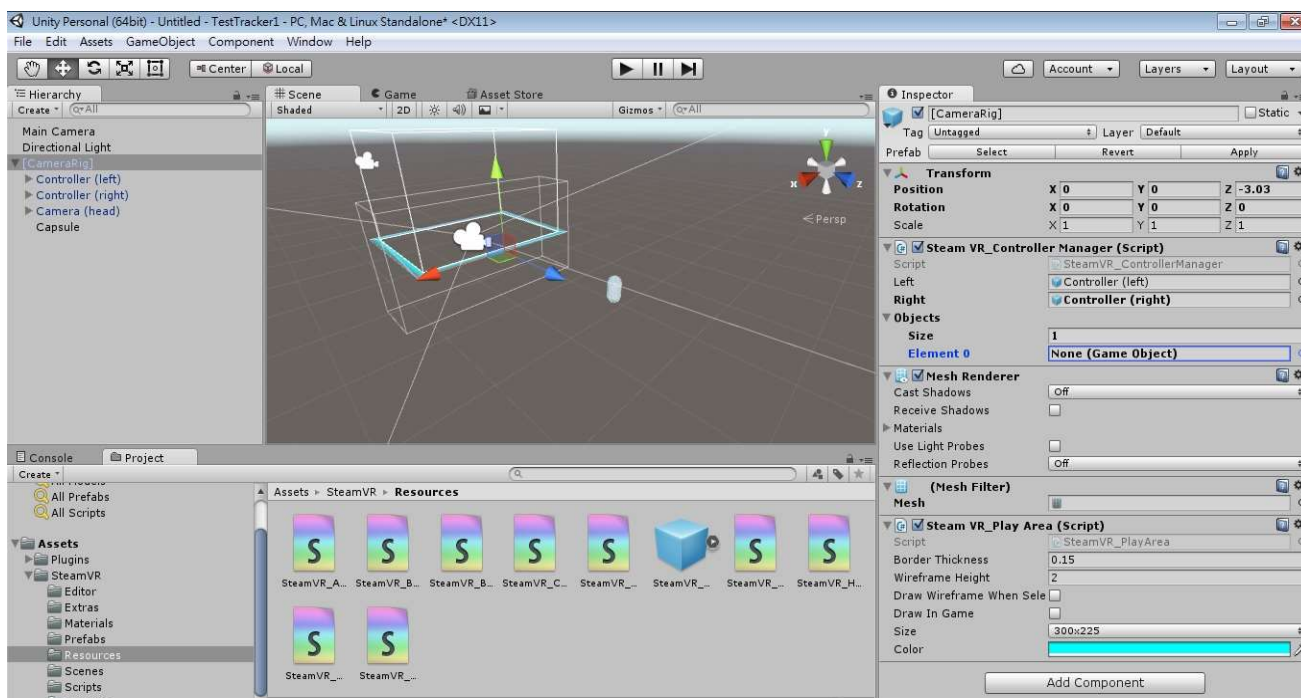
步骤2：为VIVE Tracker (3.0) 创建3D对象。本示例中使用Capsule。



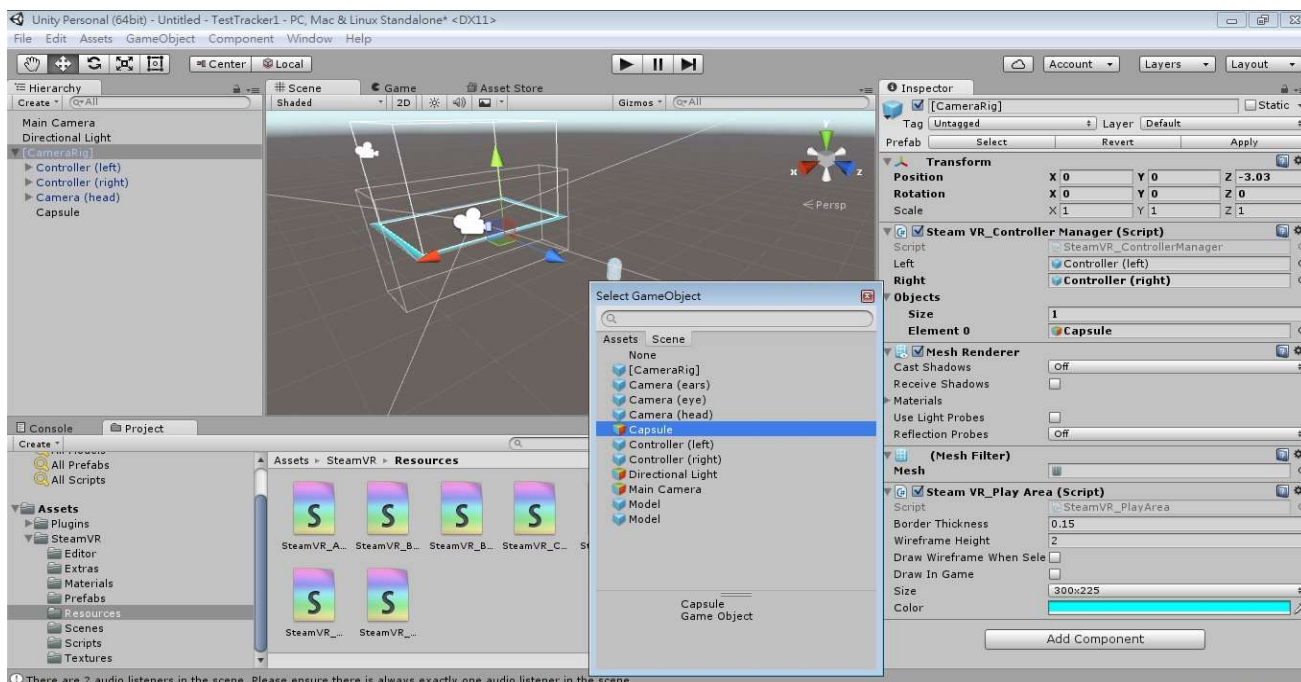
步骤3：将组件SteamVR_Tracked Object添加至3D对象胶囊中。



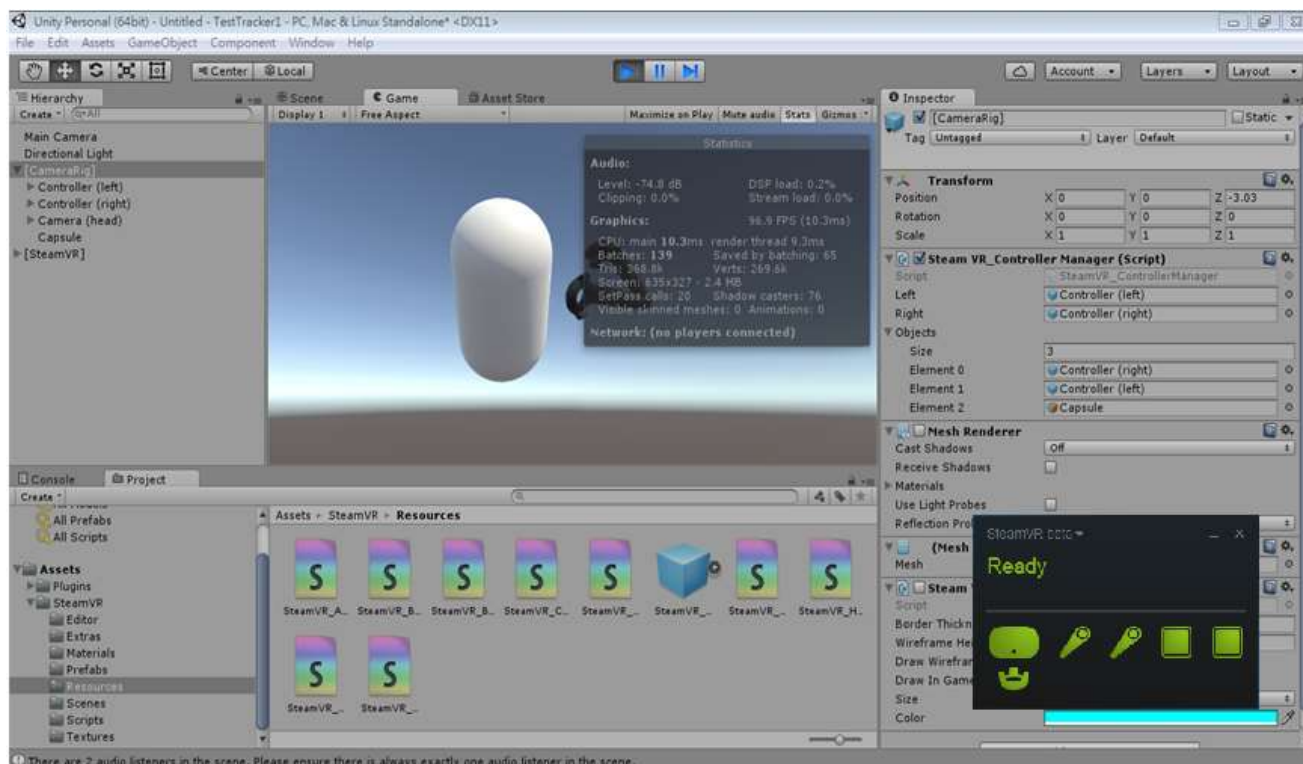
步骤4：在SteamVR控制器管理器中，设置对象项的尺寸。本示例中，设置中使用了一个VIVE Tracker (3.0)。



步骤5：在SteamVR控制器管理器中，于元素0字段选择Capsule作为GameObject。

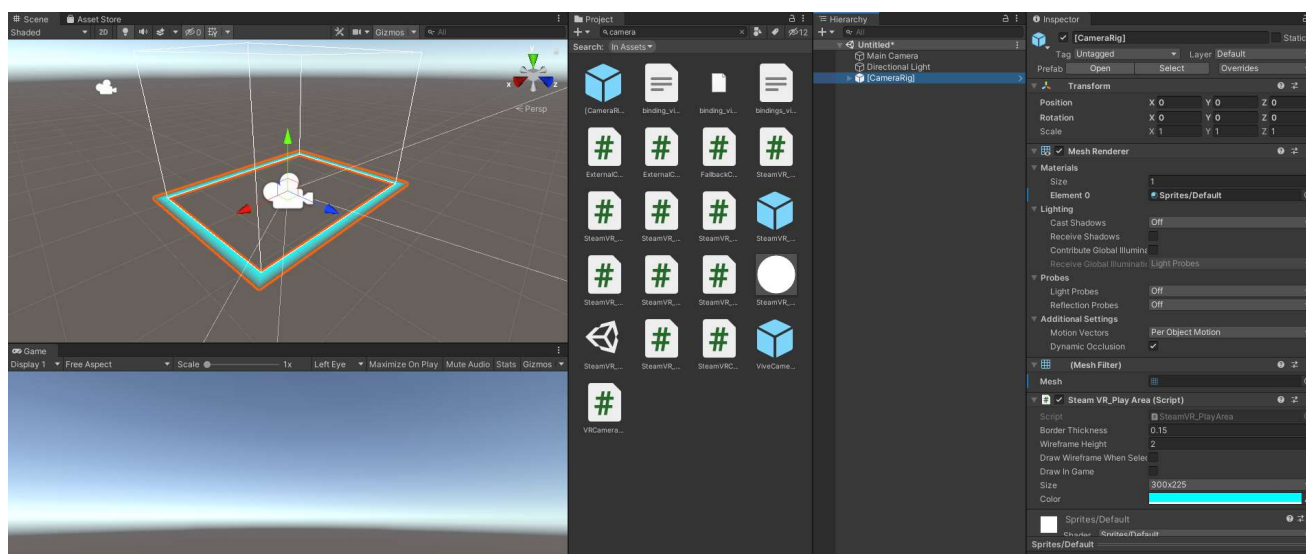


步骤6：在Unity中点击运行。当您移动VIVE Tracker (3.0) 时，您会看到胶囊对象也在内容中移动。

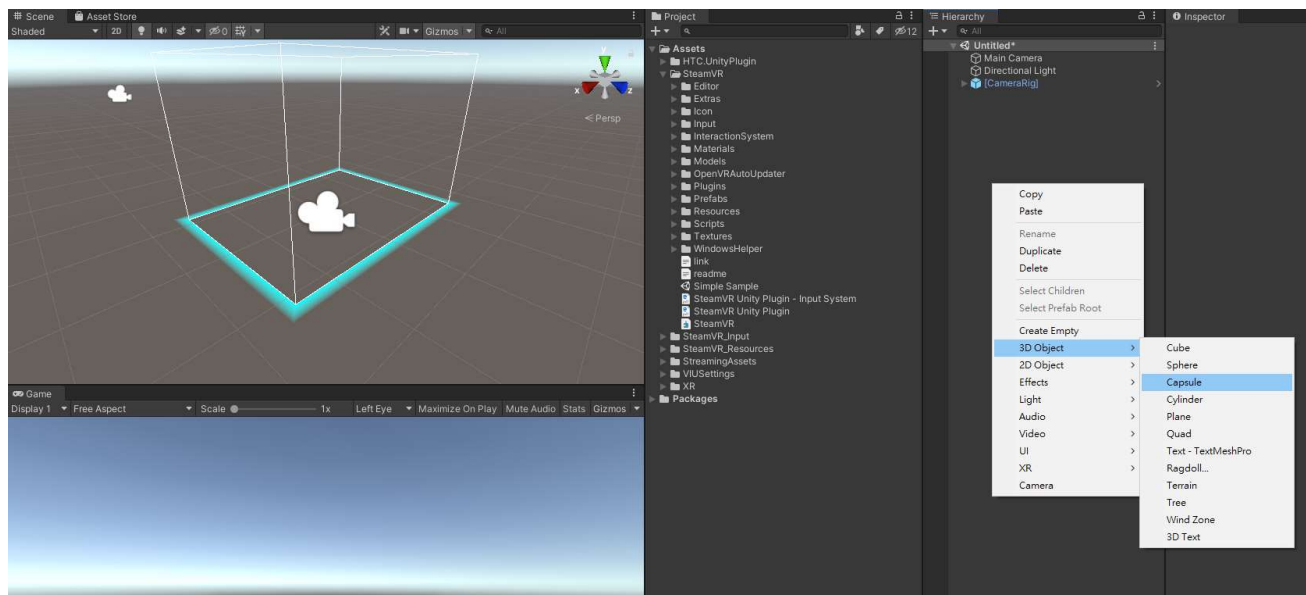


使用SteamVR 2.0插件

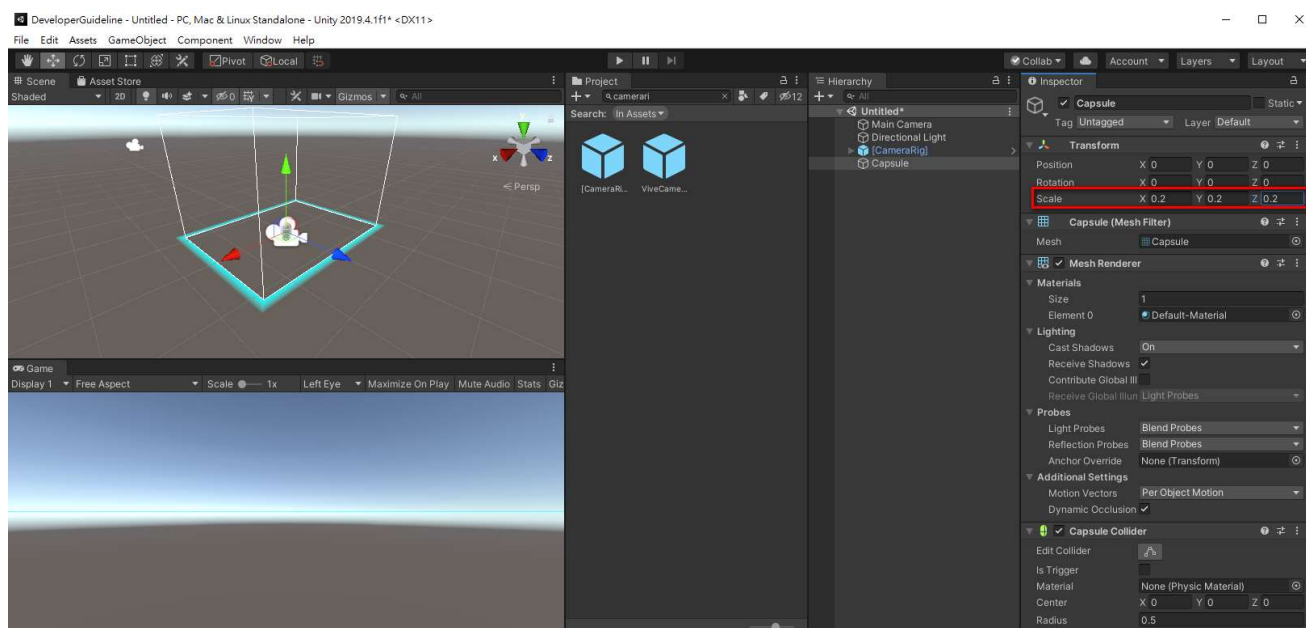
步骤1：将“CameraRig”添加到Hierarchy。



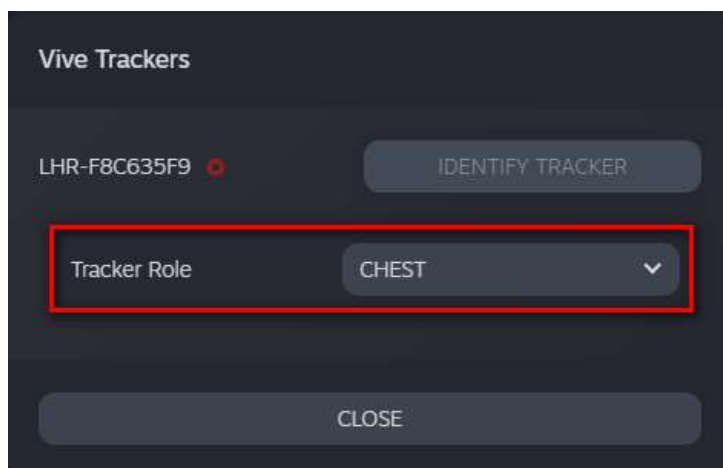
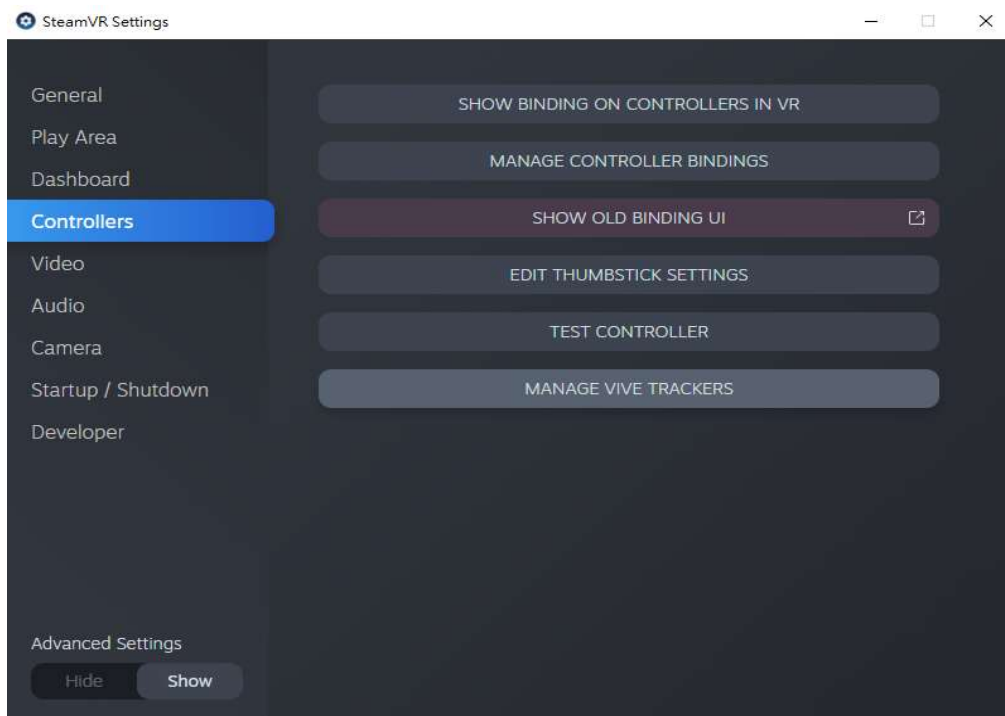
步骤2：为VIVE Tracker (3.0) 创建3D对象。本示例中使用Capsule。



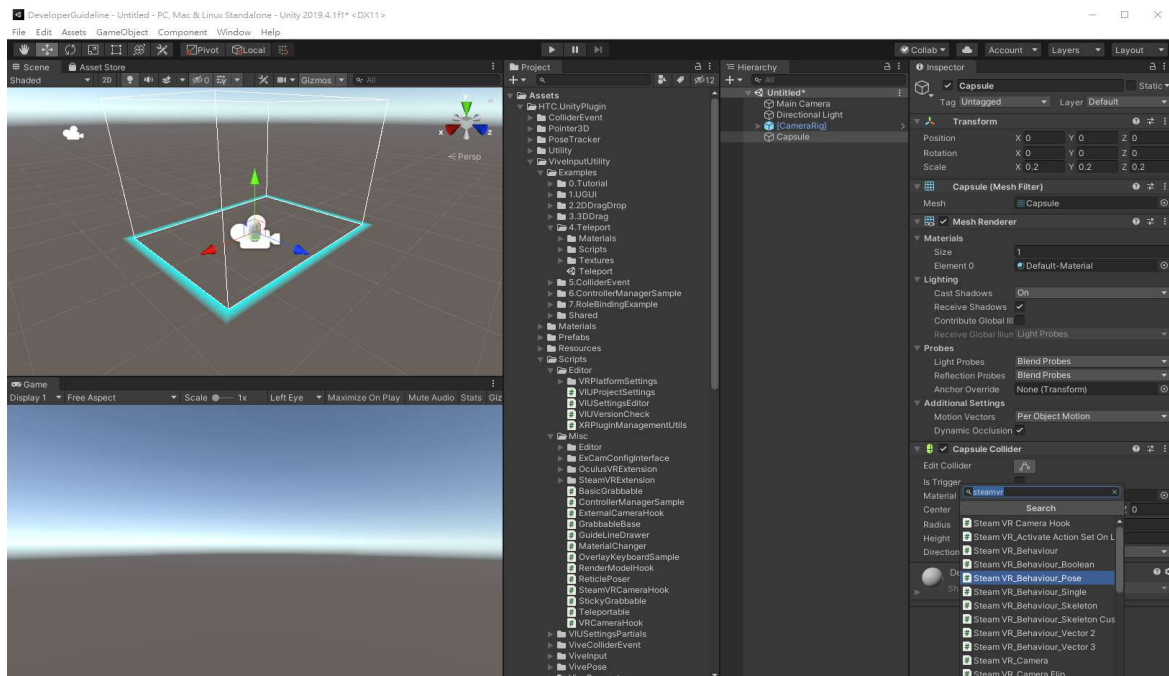
步骤3：将胶囊的缩放比例调整为0.2。



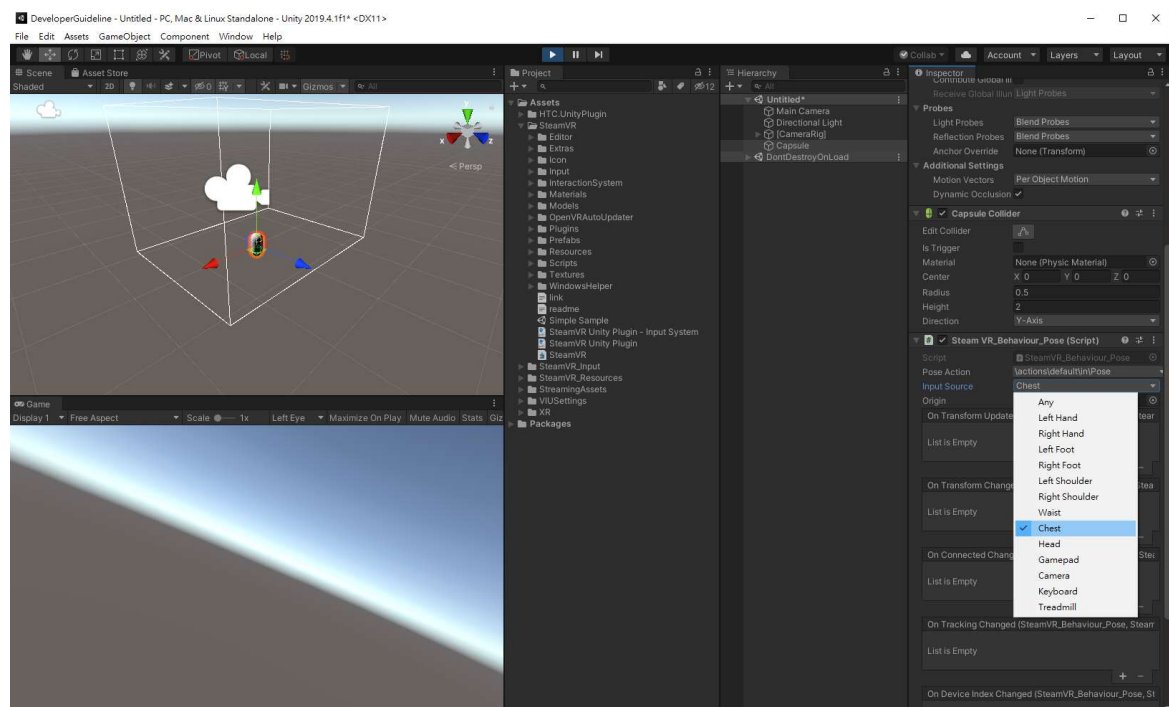
步骤4：在SteamVR中，点击设备>控制器设置>管理VIVE追踪器来编辑追踪器的角色。本例中使用的是胸部追踪器。



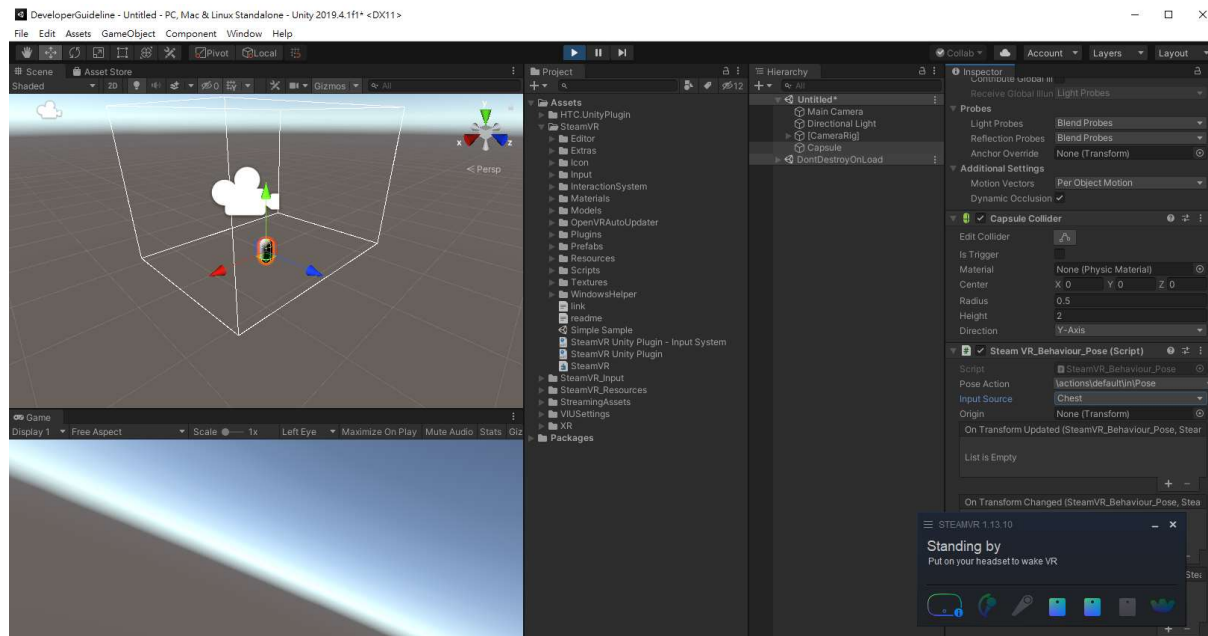
步骤5：添加SteamVR_Behaviour_Pose对象



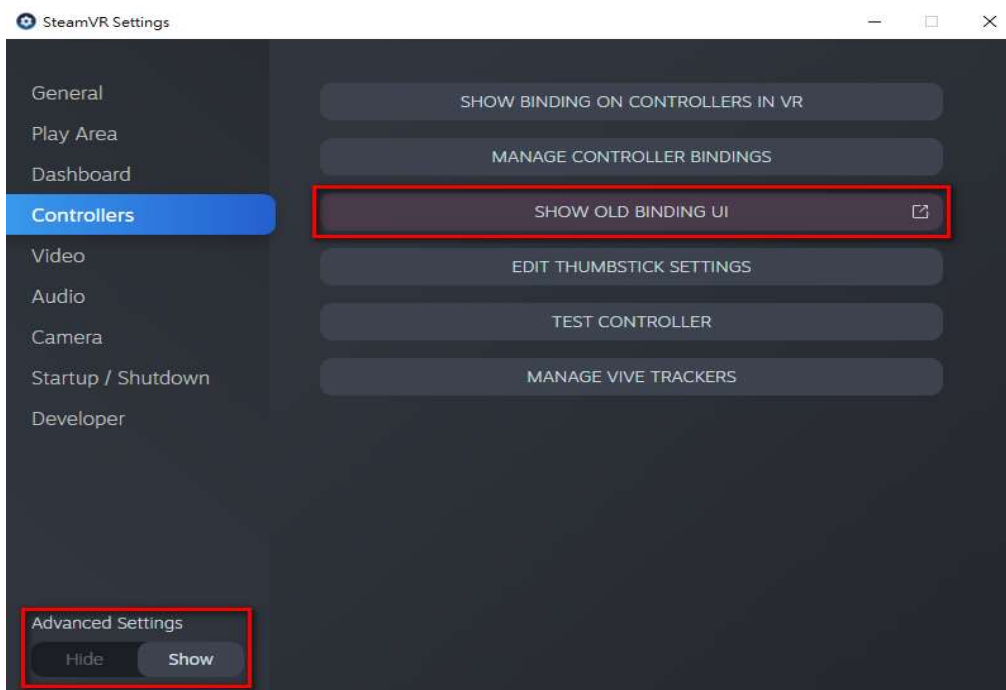
步骤6：将输入源设置为胸部。



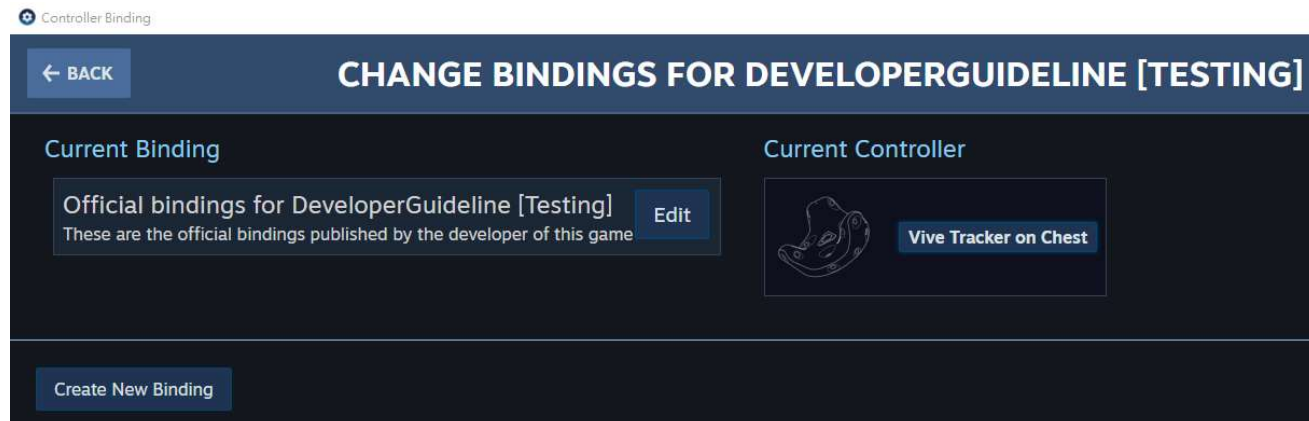
步骤7：在Unity中，点击运行。你需要为追踪器添加动作姿势。



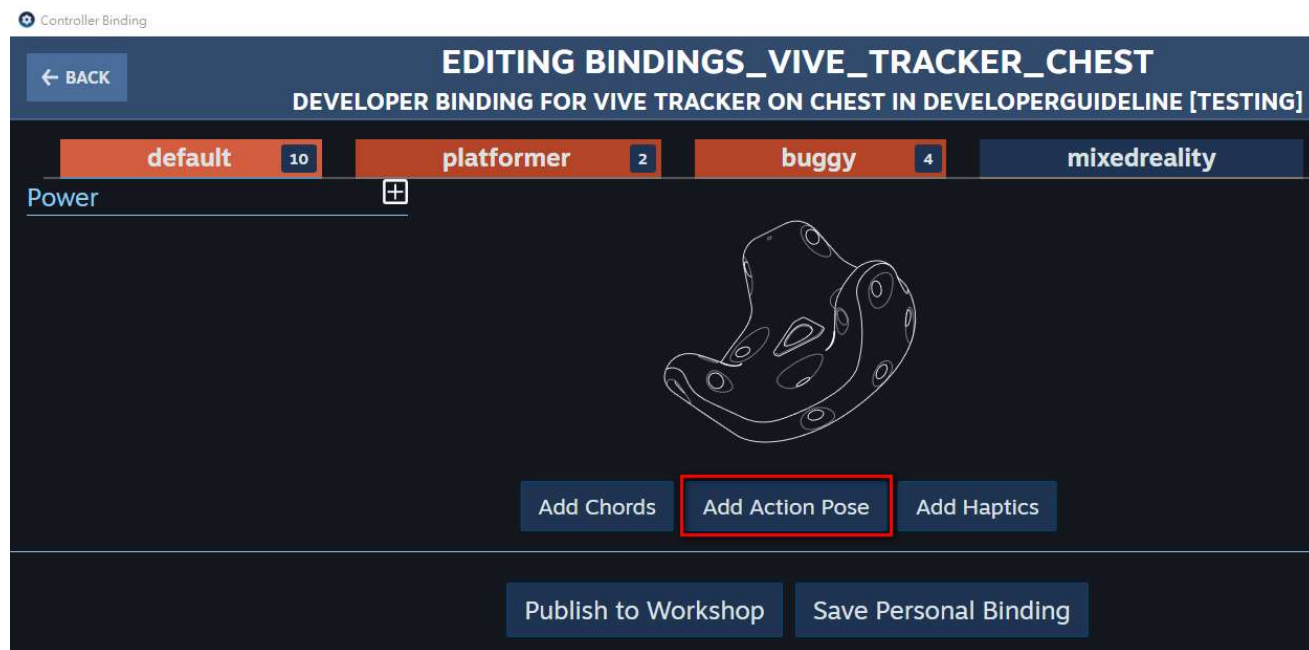
在SteamVR设置中，单击“高级设置”中的“显示”。单击“显示旧的绑定UI”，然后选择您的项目。在此示例中，使用DeveloperGuideline。



选择应用后，点击“编辑Vive Tracker胸部”按钮。

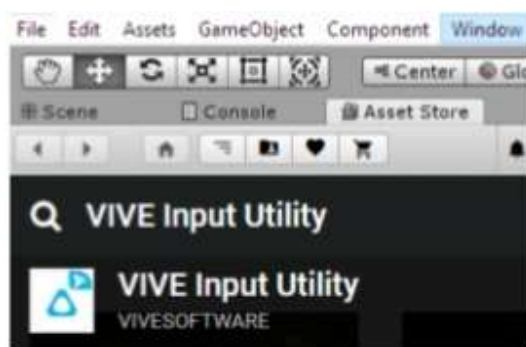


点击“添加动作姿势”按钮，将用户胸部原始姿势文件夹中的姿势从“未使用”更改为“姿势”。当您移动VIVE Tracker (3.0) 时，会发现胶囊对象也在内容中移动。

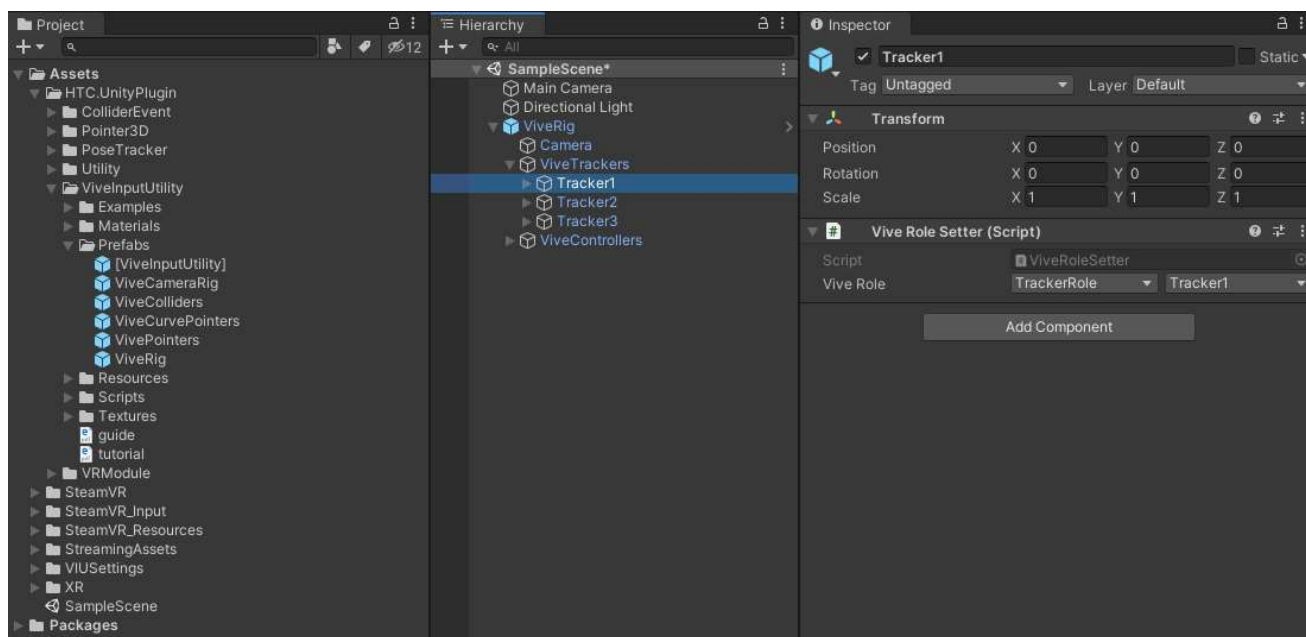


使用VIVE Unity插件

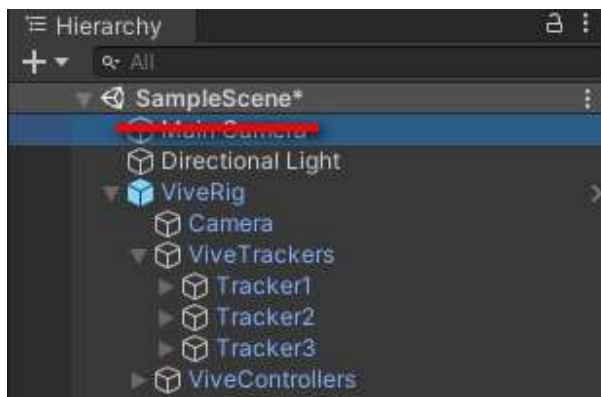
将VIVE Input Utility插件导入项目。若尚未安装，可从Unity Asset Store下载。该插件（VIU）是一款跨平台VR工具包，特别支持VIVE追踪器。



步骤1：在场景层级中添加“ViveCameraRig”或“ViveRig”，即可在Unity中支持控制器和追踪器。



步骤2. 就像 SteamVR 的 “CameraRig” 一样，你可以从场景中移除现有的 “Camera”，因为ViveRig包含一个摄像头。



步骤3. 运行以查看与控制器支持的跟踪器。如果安装了SteamVR插件，您将看到SteamVR的模型；否则，您只能看到内置模型。

VIVERig和VIVECameraRig预制件最多支持三个跟踪器。要添加更多跟踪器，只需复制一个跟踪器并重命名游戏对象。在检查器中，确保还更新了ViveRole。



SteamVR Tracker (3.0) 模型 (示例)

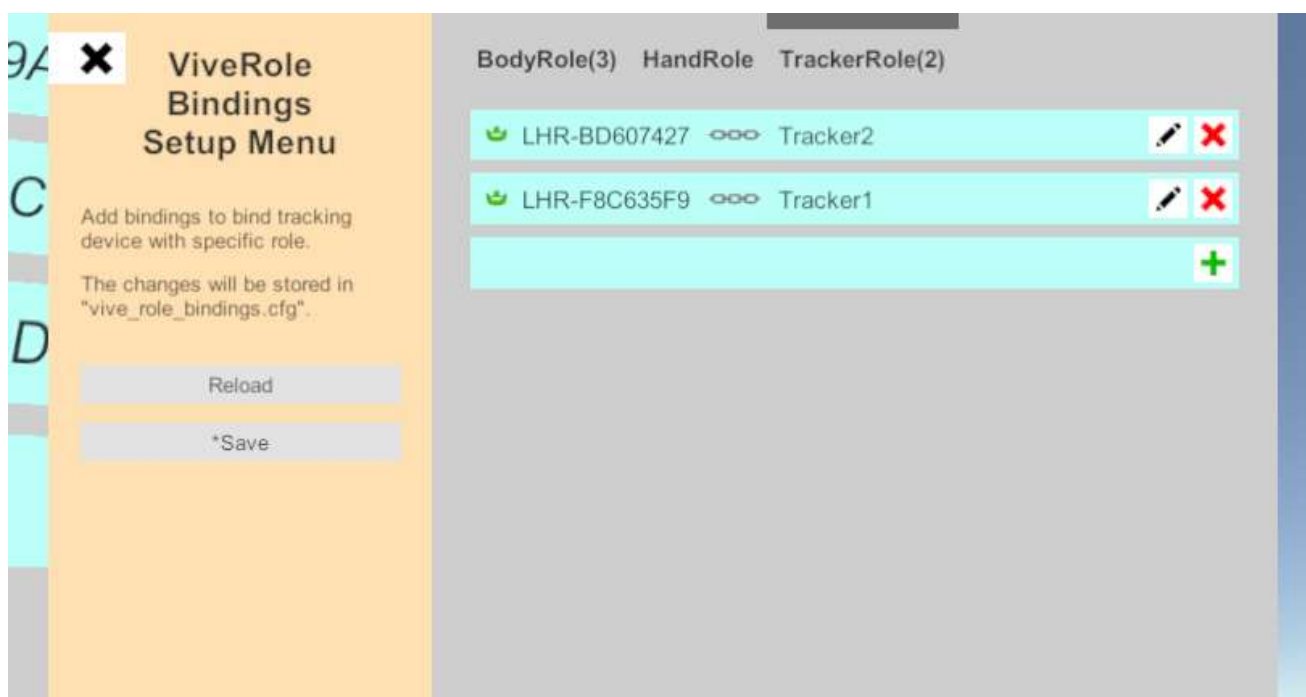
注意：目前SteamVR和VIU均支持最多11个（若不使用控制器则为13个）追踪器，总计16个设备（含头显和基站）。最近OpenVR已升级至支持64个设备，但Unity插件尚未更新（开发者可通过自定义代码实现支持）。

VIVE跟踪器和VIVE角色

使用VIVE Input Utility的一个好处是，你可以交换VIVE跟踪器或重新定义它们的附着位置，而无需担心管理设备ID和序列号。

现在可以根据上下文分配VIVE角色：跟踪器角色、身体角色、手部角色和设备角色。例如，可以使用身体角色左脚来定义始终分配给左脚的相同VIVE跟踪器。也可以为跟踪器分配相同角色，以便在电池不足时更容易更换跟踪器设备。

VIVE Input Utility还提供了一个工具，可以在不添加额外代码的情况下发现和分配角色。键盘快捷键，可设置为显示此覆盖式UI：





此外，还提供了一个API，可用于创建自定义VR用户界面来分配角色。如需了解更多信息，请参阅[_github_文章](#)。VIU插件中包含一个示例项目——“7.RoleBinding-Example”。

在Unity或Unreal Engine中跟踪

若在Unity或Unreal Engine中启用VIVE Tracker (3.0) 时遇到问题，请参考以下内容：

- 。对于Unity 3D：

- 下载VIVE Input Utility软件包的链接：AssetStore或GitHub

- o VIVE Input Utility源代码库：

- <https://github.com/VIVESoftware/VIVEInputUtility-Unity>

- 。对于Unreal Engine 4：

- o请确保您正在使用Unreal Engine 4.26或更高版本。

固件更新

请查看SteamVR通知以更新固件。您可通过以下方式更新VIVE Tracker (3.0) 固件：

1. 将HTC提供的固件二进制文件（包括MCU、FPGA和RF）复制到您电脑中“lighthouse_watchman_update.exe”文件夹下。
2. 如果VIVE控制器通过USB连接到您的计算机，请先拔掉它。
3. 使用USB数据线将一个VIVE Tracker (3.0) 连接到您的计算机。
4. 执行以下命令：

- a. 更新MCU固件：

```
lighthouse_watchman_update --target=default watchman_v3.fw
```

- b. 更新FPGA固件：

```
lighthouse_watchman_update.exe --target=default ice40_hdk_xxx.fw
```

- c. 更新射频固件：

```
lighthouse_watchman_update --target=default nrf52_xxx.fw
```